Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

DATA STRUCTURES

Abstract

Introduction to Data Structures and its Applications

Vandana M Ladwani

vandanamd@pes.edu

Contents

Introduction

Clever" ways to organize information in order to enable efficient computation

Data Structure is a way organizing data in such a way that we can perform operations on these data in an effective way for various applications

DS = Organized Data + Allowed operations

Data Structure Operations

- **Inserting**: Adding a new record to the structure.
- **Deleting**: Removing a record from the structure.
- **Traversing**: Accessing each record exactly once so that certain terms in the record may be proceeded.
- **Searching**: Finding the location of the record with a given key value.
- **Sorting**: Arranging the records either in ascending or descending order according to some key.
- **Merging**: Combining the records in two different sorted files into a single sorted file.
- **Copying**: Records in one file are copied to another file.

Classification of Data Structures

- Simple Data Structure
- Compound Data Structure
    - Linear Data Structure
    - Nonlinear Data Structure

➢ Simple Data Structure

It can be constructed with the help of Primitive Data Structure.
A **primitive data structure** is used to represent the standard data types of a particular programming language. Built in data types, arrays, pointers, structures, unions, etc. are examples of simple data structures.

➢ Compound Data Structure

Compound data structure can be constructed with the help of any one of the primitive data structure. Various operations that can be performed are decided by the designer to cater to the specific requirements of the applications.

It can be classified as

1) Linear Data Structure
2) Non-linear Data Structure

1. Linear Data Structures

In a linear data structure elements are accessed in a sequence.
Some of the operations that can be applied on linear data structure include

- Add an element
- Delete an element
- Traverse
- Sort the list of elements
- Search for a data element

Example: Stack, Queue, Tables, List, and Linked Lists.

2. Non-Linear Data Structures

Non-linear data structure can be constructed as a collection of randomly distributed set of data item. In non-linear Data structure the relationship of adjacency is not maintained between the Data items.
Some of the operations that can be applied on non-linear data structures:

- Add an element
- Delete an element
- Traverse
- Search for a data element

Example: Tree, Decision tree, Graph and Forest

Applications of Data Structures

Array

- Represent/implement other data structures in memory
- Store files in memory

Linked Lists

- Represent/implement other data structures in memory
- Dynamically allocate space in the main memory
- Allocate blocks in hard disk
- Manipulate large numbers

Stacks

- Recursion
- Call stack is used to keep track of program with multiple functions
- Infix to postfix expression conversion
- Evaluate a postfix expression
- Rearranging railroad cars
- Implement undo and redo operation for various applications

Queues

- Job scheduling
- Process scheduling in Operating system
- Handling events in event controlled applications
- Handling of interrupts by the operating system
- Store the browsing history

Trees

- Auto complete features (Trie)
- For easier substring matching
- For metadata indexing in file systems (B+ tree)
- To maintain table indices in relational database systems (B+ tree)
- Store dictionary in a mobile (Trie)

- To check spellings (Trie)
- To construct associative array (Red black trees)
- To ensure direct access of data blocks in file systems (B tree)
- Used by compilers to check the syntax of a statement in a program (Parse Trees)
- Used by operating systems to maintain the structure of a file system

➢ Heaps

- Priority queue Implementation
- Heapsort

➢ Graphs

- Social network analysis
- Shortest path problems
- Disease modelling
- Citation of journals
- Computer Networks

Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

MEMORY ALLOCATION

Abstract

Static and Dynamic Memory allocation

Vandana M Ladwani

vandanamd@pes.edu

Contents

Static and Dynamic Memory Allocation

Memory can be allocated for variables using two different techniques like static and dynamic memory allocation

Static Memory Allocation

Memory is assigned before the execution of the program begins i.e., during compilation time in the stack region. The compiler allocates the required memory space.

Advantages of Static Memory Allocation

1. Memory is allocated in a more structured area of memory, called the stack region.
2. No need of pointers to access the data
3. Faster execution than Dynamic memory allocation

Disadvantages of Static Memory Allocation

1. The memory is allocated during compilation time. Hence, the memory allocated is fixed and cannot be altered during run time.
2. This leads to under utilization of memory if more memory is allocated
3. This leads to over utilization of memory if less memory is allocated
4. Useful only when the data is fixed and known before processing
5. Memory cannot be deleted explicitly only overwriting takes place
6. If elements are to be added to or deleted from intermediate positions then shifting of elements is required to be done

Dynamic Memory Allocation

Consider a particular application for which we need to change the size of array at run time, if array is allocated statically, extending or shrinking of the memory during run time is not possible. Thus by using static memory allocation, we cannot utilize the memory efficiently instead we can use dynamic memory allocation in this scenario

The process of allocating memory at runtime is known as dynamic memory allocation. Memory is allocated or deallocated during run-time (during the

execution of the program) in the heap region. Library routines defined in **stdlib.h** known as "memory management functions" are used for allocating and releasing also termed as freeing memory during execution of a program. These functions are as follows

malloc(): Allocates requested size of bytes and returns a void pointer pointing to the first byte of the allocated space

calloc(): Allocates space for an array of elements, initialize them to zero and then return a void pointer to the memory

realloc(): Modifies the size of previously allocated space using above functions

free(): Releases the allocated memory

Advantages of Dynamic Memory Allocation

1. Memory is allocated only when the program unit is active.
2. Memory is utilized efficiently.
3. Allocated memory can be resized during run time based on the dynamic requirement of the user/program.

Disadvantages of Dynamic Memory Allocation

1. Memory is allocated in the heap region which is less structured.
2. Execution is comparatively slower.
3. Pointers are required to work with dynamically allocated memory region.

Difference between static and Dynamic Memory Allocation

Static Memory Allocation)	Dynamic Memory Allocation
The memory is allocated during compile time.	The memory is allocated during run time
The size of the memory to be allocated is fixed during compile time and cannot be altered during run time.	As and when memory is required, memory can be allocated. If memory is not required it can be deallocated.
Memory is allocated in stack area	Memory is allocated in heap area
Used only when the data size is fixed and known in advance before processing	Used for unpredictable memory requirement
Execution is faster, since memory is already allocated and data manipulation is done on these allocated memory locations	Execution is slower since memory has to be allocated during run time. Data manipulation is done only after allocating the memory
Example: Arrays	Example: Linked List

Dynamic Memory Management Functions

malloc()

Prototype: void *malloc(size_t size);
Description: malloc() function allocates size bytes and returns a pointer to the allocated memory. The memory is not initialized. If size is 0, then malloc() returns either NULL, or a unique pointer value. The malloc() function returns a void pointer to the allocated memory. On error, function return NULL.

calloc()

Prototype: void *calloc(size_t num_blocks, size_t size);
Description: The calloc() function allocates memory for an array of num_blocks elements of size bytes each and returns a pointer to the allocated memory. The memory is set to zero. If num_blocks or size is 0, then calloc() returns either NULL, or a unique pointer value. The calloc() function return a void pointer to the allocated memory.

realloc()

Prototype: void *realloc(void *ptr, size_t size);
Description: This function changes the size of the memory block pointed to by ptr to size bytes. The contents will be unchanged in the range from the start of the region up to the minimum of the old and new sizes. If the new size is larger than the old size, the added memory will not be initialized. If ptr is NULL, then the call is equivalent to malloc(size) for size>0; if size is equal to zero, and ptr is not NULL, then the call is equivalent to free(ptr). If the area pointed by ptr gets moved, a free(ptr) is done.The realloc() function returns a pointer to the newly allocated memory, which is suitably aligned for any kind of variable and may be different from ptr, or NULL if the request fails. If realloc() fails the original block is left untouched, and it is not freed or moved.

free()

Prototype: void free(void *ptr);
Description: The free() function frees the memory space pointed to by ptr, which must have been returned by a previous call to malloc(), calloc() or realloc(). Otherwise, or if free(ptr) has already been called before, undefined behaviour occurs. If ptr is NULL, no operation is performed. The free() function returns no value.

Differences between malloc and calloc

malloc()	calloc()
stands for memory allocation	Stands for contiguous allocation
This function takes single argument. Syntax: void *malloc(size_t size); size: total number of bytes to be allocated	This function takes two arguments Syntax: void *calloc(size_t n, size_t size); n :number of blocks to be allocated. Size: number of bytes to be allocated for each block
Allocates a block of memory of size bytes	Allocates multiple blocks of memory, each block with the same size
Allocated space will be initialized to junk values	Each byte of allocated space is initialized to zero
malloc is faster than calloc.	calloc takes little longer(not recognizable in practical scenario) than malloc because of the extra step of initializing the allocated memory.

Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

SINGLY LINKED LIST

Abstract

Singly linked list overview with its advantages and disadvantages. Types of singly linked list and pseudo code for various operations

Vandana M Ladwani
vandanamd@pes.edu

PES UNIVERSITY

Contents

Singly Linked list

Overview

Linked lists and arrays are similar in the sense that they both store collections of data. Array is the most common data structure used to store collections of elements. Arrays are convenient to declare and provide the easy access to elements through index. Once the array is set up, access to any element is convenient and fast.

Disadvantages of Array

- The size of the array is fixed. Most often this size is specified at compile time. This makes the programmers to allocate arrays, which seems "large enough" than required.
- Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.
- Deleting an element from an array is not possible, deletion just overwrites the content in the memory location that too at the expense of shifting elements

Linked lists have their own strengths and weaknesses, but they happen to be strong where arrays are weak. Array allocates the memory for all its elements in one block whereas linked lists use an entirely different strategy. Linked list allocates memory for each element separately and only when necessary. A linked list is a non-sequential collection of data items. It is a dynamic data structure. For every data item in a linked list, there is an associated pointer that would give the memory location of the data item/items to which it is linked in the list. The data items in the linked list are not in consecutive memory locations. They may be anywhere, but the accessing of these data items is easier as each data item contains the address of the linked data item.

Advantages of linked lists

Linked lists have many advantages. Some of the very important advantages are:

1. Linked lists are dynamic data structures. i.e., they can grow or shrink during the execution of a program.
2. Linked lists have efficient memory utilization. Here, memory is not pre-allocated. Memory is allocated whenever it is required and it is de-allocated (released) when it is no longer needed.

3. Insertion and Deletions are easier and efficient. Linked lists provide flexibility in inserting a data item at a specified position and deleting a data item from the given position.
4. Linked list is used as an important data structure for implementing many complex applications

Disadvantages of linked lists

1. It consumes more space because every node requires an additional pointer to store address of the link node.
2. Searching a particular element in list is difficult and also time consuming.

Types of Linked Lists

Basically we can put linked lists into the following four items:

1. Singly Linked List.
2. Doubly Linked List.
3. Circular Singly Linked List.
4. Circular Doubly Linked List.

A singly linked list is the one in which all nodes are linked together in sequential manner. Each node holds a single pointer which points to the next item. Hence, it is also called as linear linked list.

A double linked list is one in which all nodes are linked together by two links which helps in accessing both the successor node (link node) and predecessor node (previous node) from any arbitrary node within the list. Therefore each node in a double linked list has two link fields (pointers) to point to the left node (previous) and the right node (link). This helps to traverse in forward direction and backward direction.

A circular linked list is one, which has no beginning and no end. A singly linked list can be made a circular linked list by simply storing address of the very first node in the link field of the last node.

A circular doubly linked list is one, which has both the successor pointer and predecessor pointer and last node and first node are connected

Comparison between array and linked list:

ArrayList	Linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient	Insertions and Deletions are efficient
Elements are in contiguous memory locations	Elements are not in contiguous memory locations
May result in memory wastage if all the allocated space is not used	Since memory is allocated dynamically (as per requirement) there is no wastage of memory.
Sequential and random access is faster	Sequential and random access is slow

Singly Linked List

A linked list allocates space for each element separately in a block of memory called “node". The list gets an overall structure by using pointers to connect all its nodes together like the links in a chain. Each node contains two fields; a "data" field to store whatever element, and a "link" field which is a pointer used to link to the link node. Each node is allocated in the heap using malloc(), so the node memory continues to exist until it is explicitly de-allocated using free(). Head/start is a pointer to the first node in the linked list.

The basic operations in a singly linked list are:

- Creation.
- Insertion.
- Deletion.
- Traversing
- Reversing a list
- Concatenating two lists

Creating a node

- Get a node using malloc
- If memory is allocated successfully
    1. Set data part
    2. Set link part

Insertion of a Node

Memory is to be allocated for the new node. The new node will contain empty data field and empty link field. The data field of the new node is set to the value given by the user. The link field of the new node is assigned to NULL. The new node can then be inserted at three different places namely:

- Inserting a node at the beginning.
- Inserting a node at the end.
- Inserting a node at intermediate position.
- Inserting a node at the beginning:

Inserting a node at front

- Get the new node using createnode().
- new_node = createnode();
- If the list is empty then head = new_node.
- If the list is not empty, follow the steps given below:
  - new_node -> link = head;
  - head = new_node;

Inserting a node at the end

- Get the new node using createnode()
  new_node = createnode();
- If the list is empty then head = new_node.
- If the list is not empty follow the steps given below:
  - temp = head;
  - while(temp -> link != NULL)
    - temp = temp -> link;
    - temp -> link = new_node;

Inserting a node at intermediate position

- Get the new node using createnode().
  new_node = createnode();
- If the position is greater than length of linked list+1, specified position is invalid.
- Store the heading address (which is in head pointer) in temp and prev pointers. Then traverse the temp pointer unto the specified position followed by prev pointer.
- After reaching the specified position, follow the steps given below:
  If position is 1

Insert at front

If position=length of linked list+1

Insert at end

Else

if intermediate position

- prev -> link = new_node;
- new_node -> link = temp;

Deletion of a node

Another primitive operation that can be done in a singly linked list is the deletion of a node. Memory is to be released for the node to be deleted. A node can be deleted from the list from three different places namely.

- Deleting a node at the beginning.
- Deleting a node at the end.
- Deleting a node at intermediate position.
- Deleting a node at the beginning:

Deleting a node from front of linked list

If list is empty then display 'Empty List' message.

- If the list is not empty, follow the steps given below:

```
temp = head;
head = head -> link;
free(temp);
```

Deleting a node at the end

The following steps are followed to delete a node at the end of the list:

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given below:

```
temp = prev = head;
while(temp -> link != NULL)
{
 prev = temp;
 temp = temp -> link;
}
prev -> link = NULL;
free(temp);
```

Deleting a node at Intermediate position

- If list is empty then display 'Empty List' message
- If the list is not empty (has atleast two nodes), follow the steps given below.

if(pos > 1 && pos < nodectr)

- temp = prev = head;
- ctr = 1;
- while(ctr < pos)
    - prev = temp;
    - temp = temp -> link;
    - ctr++;
- prev -> link = temp -> link;
- free(temp);

Traversal and displaying a list (Left to Right)

To display the information, you have to traverse (move) a linked list, node by node from the first node, until the end of the list is reached.

Traversing a list involves repeating following steps till head becomes NULL

- Assign the address of head pointer to a temp pointer.
- Display the information from the data field of each node
- Advance head pointer

PES University
Department of Computer Science and Engineering

UE19CS202- Data Structures and its Applications(4-0-0-4-4)

UNIT - 1 : INTRODUCTION, STATIC AND DYNAMIC MEMORY ALLOCATION, LINKED LIST AND RELATED CASE STUDY

Note : Pointers, Arrays, Structures to be revised for clear understanding.

Unit 1: Overview

Static and Dynamic Memory Allocation, Singly Linked List.**Linked List:** Doubly Linked List, Circular Linked List – Single and Double, Multilist : Introduction to sparse matrix (structure). **Application:** Case Study -**Text Editor , Assembler**- Creation of a Symbol Table.

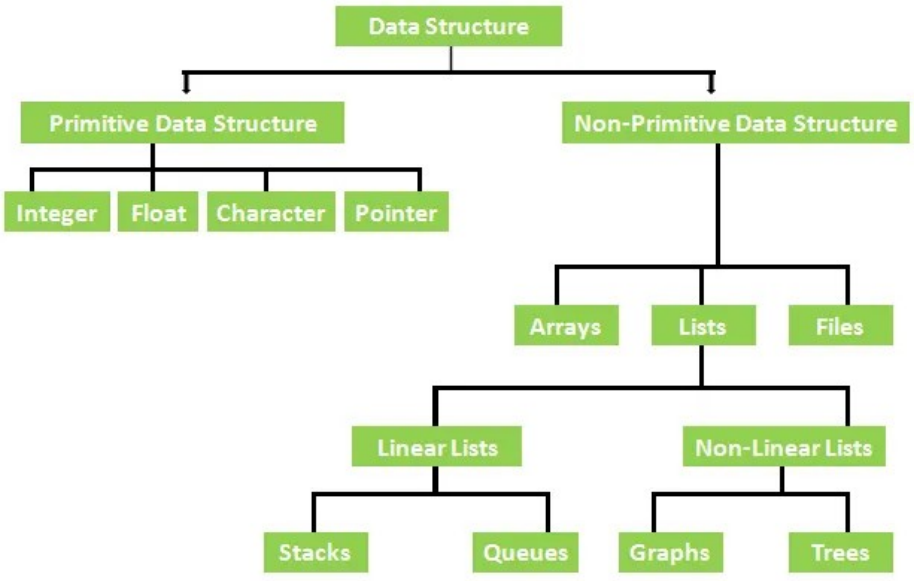
Contents :

Introduction :

- Data Structures is a way of organizing ,managing and storing the data that provides a way for a collection of different data values and the relationship among them.
- Choice of the data structure begins with the choice of Abstract Data Type

- Example : Arrays used to store list of elements having the same type, Structures are used to store the list of elements having different data types
- Other data Structures includes - Linked, Stacks, Queues, Trees, Graphs which will be discussed in detail throughout this course.
- Each of these data structures have a special way of organizing so that users can choose the data structure based on the requirement.
- In other words Data Structure is an arrangement of data in computer's memory in such a way that it could make the data quickly available to the processor for required calculations
- Data Structures should be seen as a logical perspective should address the 2 primary concerns:
    - How will the data be stored?
    - What operations will be performed on it?
- Since data structure is a way of organizing the data, so the functional definition of a data structure should be independent of its implementation.
- The functional definition of a data structure is known as ADT (Abstract Data Type) which is independent of implementation.
- The way in which the data is organized affects the performance of a program for different tasks.
- Programmers decide which data structures are to be used based on the nature of the data and the processes that need to be performed on that data.

Types of Data Structures



- Data Structures are classified into 2 types:
    - Primitive Data Structures

        - All primitive data types are primitive data structures
        - THey follow the machine instructions
    - Non Primitive Data Structures
        - Are built using primitive data types.
- **Linear Data Structures :** It is a type of data structure where the data elements are accessed in a sequential manner. However the elements can be stored in any order. Elements can be traversed in a single run. **Examples** - Arrays, Lists, Stacks
- **Non Linear Data Structures** : It is a type of structure where there is no physical adjacency between the elements, Elements can be accessed in a non-sequential manner. **Examples** - Trees, Graphs.

Data Types Vs Data Structures

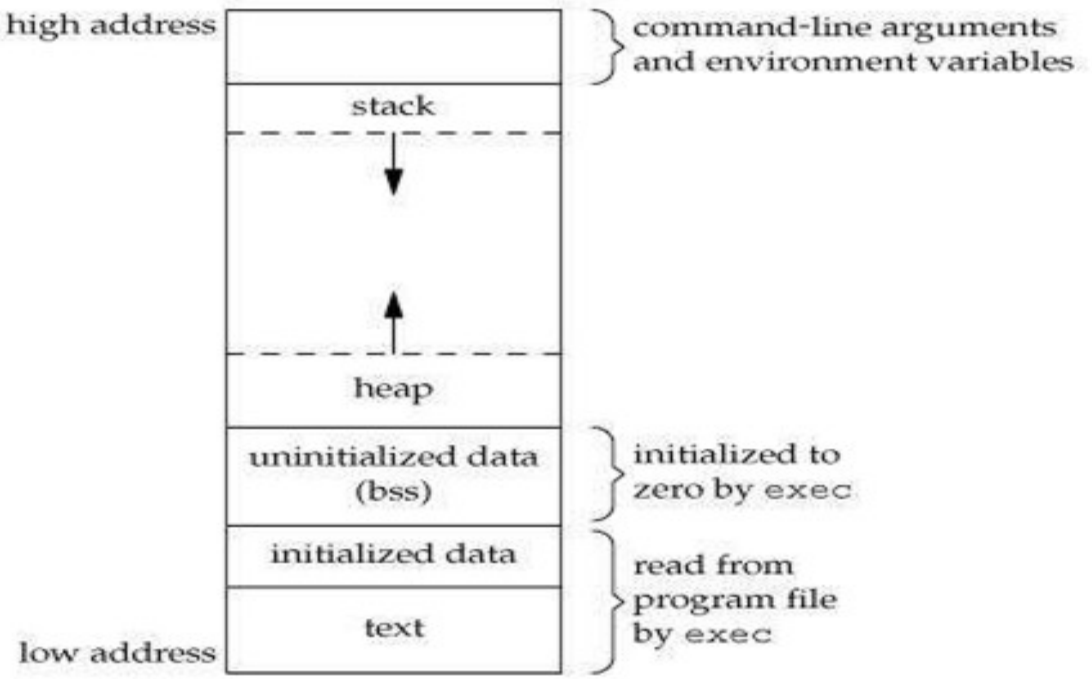
Data Type	**Data Structure**
Data type is a source to hold the variable	Collection of different kinds of data
Implementation - Abstract	Implementation - Concrete
Can hold data values	Can hold within a single object
Values can be directly assigned to types	Data assigned to operations through operations like - push, pop etc..
Example - int, float, char	**Example - Stack, queue, trees**

Abstract Data Type:

- It is a logical description of how the data and the operations are viewed without knowing how they will be implemented.
- Concerned only with what data is representing and not with how it will eventually be constructed.
- Provides level of abstraction and encapsulation around the data.
- The idea is that by encapsulating the details of the implementation,data is getting hidden from user view. This is called information hiding.
- The implementation of an abstract data type, often referred to as a data structure, will require that we provide a physical view of the data using some collection of programming constructs and primitive data types.

Memory Management of C Programming

When a C program is executed, it's the executable image loaded into RAM in an organized manner.



Consider the figure above: Memory is divided into various segments as described below:

Text Segment :

- Text segment contains machine code of the compiled program.
- Usually, the text segment is sharable so that only a single copy needs to be in memory for frequently executed programs, such as text editors, the C compiler, the shells, and so on.
- The text segment of an executable object file is often read-only segment that prevents a program from being accidentally modified

Initialized Data Segment:

- Stores global, static, constant, and external variables that are initialized beforehand.
- Data segment is not read-only, since the values of the variables can be altered at run time
- #include<stdio.h>
  Char s[]= “PES”;
  Char c[]=”University”;
  Int main()
  {
  Static int i = 11;
  }

Uninitialized Data Segment:

- Data in this segment is initialized to arithmetic 0 before the program starts executing.
- Uninitialized data starts at the end of the data segment and contains all global variables and static variables that are initialized to 0 or do not have explicit initialization in source code.
- #include<stdio.h>
  Char c;
  Int main()
  {
  Int i;
  Static int j;
  }

Heap:

- Heap is the segment where *dynamic memory allocation* usually takes place.
- When some more memory needs to be allocated using malloc and calloc function, heap grows upward.
- The Heap area is shared by all shared libraries and dynamically loaded modules in a process.

Stack:

- Stack segment is used to store all local variables and is used for passing arguments to the functions along with the return address of the instruction which is to be executed after the function call is over.
- Local variables have a scope to the block which they are defined in, they are created when control enters into the block.
- Activation Records of Function calls are created in the stack.

Memory Management : Static and Dynamic Memory Allocation

- Memory allocation in programming is very important for storing values when you assign them to variables.
- The allocation is done either before or at the time of program execution.

Static Memory Allocation

- In static memory allocation, memory is allocated while the program is being compiled : Compile Time
- Each static or global variable defines one block of space, of a fixed size. The space is allocated once, when your program is executed(part of the exec operation), and is never freed.
- Memory size will be fixed and cannot be changed or modified.
- Memory is allocated in the stack..
- Example : Arrays : Declare static array to stored integer data, when we declare array of 500, required memory = 500*2 = 1000 bytes on 32 bit operating system and required memory = 500*4 = 2000 on a 64 bit operating system. 1000 (on Windows) and 2000 (on Linux) has reserved for the above declared array and we could not use that memory, even if that array contains only one integer or no data. Suppose if an array contains 500 integers, we have deleted 499 still we could not use that memory for other purposes.

Advantages of Static Allocation

- Allocation speed is faster.
- No extra algorithm needed.
- No fragmentation problem required to achieve allocation task.

Disadvantages of Static Allocation

- No memory reusability.
- Less efficient allocation.
- Memory once allocated cannot be reallocated.

Dynamic Memory Allocation

- In dynamic memory allocation, memory is allocated during the execution of the program - Run Time.
- Memory size can be modified while executing the program.
- Dynamic memory allocation manage an area of process Virtual memory Area called Heap
- Example : Linked List.
- ‘C’ offers 4 Dynamic memory allocation functions defined in stdlib.h- malloc(), calloc(), realloc(), free().

1. **malloc() - Memory Allocation**
    - Allocate a single large block of memory with the specified size.

- It returns a pointer of type void which can be cast into a pointer of any form.
- It initializes each block with default garbage value.
- Syntax: ptr = (type*)malloc(sizeof(bytesize));
- Example : ptr=(int*)malloc(100*sizeof(int));
- In the above example, int occupies 4 bytes in memory, so the total memory occupied by ptr is 100*4 = 400 bytes. I.e a large memory block of size 400 bytes will be dynamically allocated to ptr.

2. **calloc() - Contiguous allocation**
    - Allocate the specified number of blocks of memory of the specified type.
    - It initializes each block with a default value '0'.
    - Syntax:ptr=(type*)calloc(n, sizeof(elementype));
    - Example : ptr = (float*) calloc(25, sizeof(float));
    - This allocated 25 continuous memory blocks of size of float(4 bytes).

3. **free()**
    - Deallocate the memory.
    - The memory allocated using functions malloc() and calloc() is not deallocated on their own.
    - Hence the free() method is used, whenever the dynamic memory allocation takes place.
    - It helps to reduce wastage of memory by freeing it.
    - Syntax : free(ptr);
    - Note : C does not support automatic garbage collection like java.

4. **realloc()- Reallocation**
    - Change the memory allocation of a previously allocated memory.
    - If the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to dynamically re-allocate memory.
    - Re-allocation of memory maintains the already present value and new blocks will be initialized with default garbage value.
    - Syntax : realloc(ptr,newsize)

- Ptr is now reallocated with a new size.

Advantages of Dynamic Memory Allocation

1. Can create additional storage whenever required
2. Can delete allocated storage whenever required.
3. Allocates memory during run time
4. Data structure can grow and shrink whenever needed.

Disadvantages of Dynamic Memory Allocation

1. Accessing heap is costly
2. Requires more time as memory is allocated during runtime.
3. Creates problems like - segmentation fault, dangling pointer issue, memory leak

LINKED LIST

- Linear Data Structure where each element is a separate object.
- Each element is called a node comprising 2 fields - data field and link field which is a reference to the next node.
- Last node has the 'null' reference.
- The entry point into a linked list is called the **head** of the list.
- Head is not a separate node, but it implicitly refers to the first node.
- If the list is empty then the head has a null reference.
- Linked list is a dynamic data structure, size of the node is not fixed, it can grow and shrink

Difference Between Arrays and Linked List

1. **Array** - a collection of similar type data elements
   Linked list -a collection of unordered linked elements known as nodes.

2. **Array**- traversal through indexes
   linked list - traversal through the head until we reach the node.

3. **Array** - Elements are stored in contiguous address space
   Linked List - Elements are at random address spaces

4. **Array** - Access is faster
   Linked List - Access is slower

5. **Array**- Insertion and Deletion of an element is not that efficient
   LInked List - Insertion and Deletion of an element is efficient

6. **Array** - Fixed Size.
   Linked List - Dynamic Size.

7. **Array** - memory assigned during compile time/ static allocation
   Linked List - Memory assigned during runtime / dynamic allocation

Types of Linked List

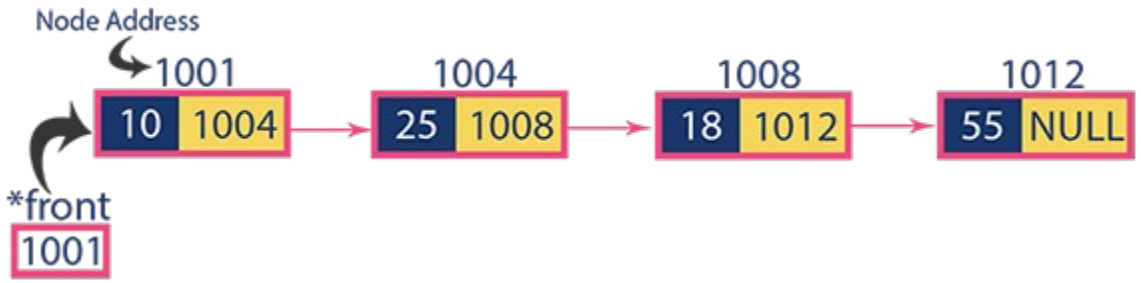
1. Singly Linked List
2. Doubly Linked List
3. Circular List

Singly Linked List

- A singly linked list is a type of linked list that is unidirectional, that is, it can be traversed in only one direction from head to the last node
- Each element in a singly linked list is called a Node.
- A Node contains the data , and a pointer to the next node which helps in maintaining the list structure
- Node structure is defined as:

```
struct node
{
  int data;
  struct node* next;
};
```

- Linked List is pictorially represented as shown below.



In the above figure, a list contains data elements as 10,25,18,55 and 1004, 1008, 1012, are the address of the next consecutive nodes. First node has a data

element as 3 and the address of the first node is stored in the next consecutive node.

Note : Head is only a reference to the first node.

- Head points to the first node of the list and it helps traverse, access other nodes in the list.
- Last node points to NULL and helps to determine the end of the list.

LIST as Abstract Data Type

- List as ADT holds the collection of nodes, which can be accessed only in sequential order.
- Nodes are connected to the next node and/or with the previous one, this gives the linked effect.
- If nodes are connected with only the next pointer the list is called **Singly Linked List - A -> B-> C-> D**
- and it is connected by the next and previous the list is called **Doubly Linked List - A <-> B<-> C<-> D.**

Note : Conventions Used for All Algorithms

- p = pointer variable of type struct node that points to the first node(head).
- temp = new node of type struct node for creating linked list.
- q = iterator of type struct node for traversing till end of list
- i = counter variable of int type, increments as we traverse every node in the list (used for position insertion and deletion to Check the position is correct).

ADT (Interface) - OPERATION PERFORMED ON A SINGLY LINKED LIST

1. Insertion
2. Deletion

Insertion: Insertion operation in a Singly Linked List can be performed in 3 ways:

1. Inserting At Beginning of the list :
    a. void insertbeginning(struct node *temp, int data, struct node *next);

2. Inserting At End of the list :
    a. void insertend(struct node *temp, int data, struct node *next);

3. Inserting At Specific location in the list :
    a. void insertposition(struct node *temp, int data, struct node *next);

Before any operation is performed, the following must be done:

Step 1 - Define a Node structure with two members data and next

Step 2 - Define a Node pointer 'head' and set it to NULL.

Step 3 - Implement the main method by displaying the operations menu and make suitable function calls in the main method to perform user selected operations

1. INSERT AT THE BEGINNING OF THE LIST

Step 1 - Create a new node **temp,** allocate memory dynamically and assign with a value.

Step 2 - Check whether list is Empty **(*p == NULL)**

Step 3 - If it is Empty then, set **temp→next = NULL and *p = temp.**

Step 4 - If it is Not Empty then, set **temp→next = *p and *p = temp**

1. INSERT AT THE END OF THE LIST

Step 1 to 3 // same as front insertion

Step 4 - If it is Not Empty then, define a node pointer q and initialize with head.

Step 5 - Keep moving the q to its next node until it reaches to the last node in the list (until **q → next = NULL).**

Step 6 - Set **q → next = temp.**

2. INSERT AT A SPECIFIC POSITION IN THE LIST

Step 1 - Create a new node temp with given value.

Step 2 - Check whether list is Empty(*p == NULL)

Step 3 - If it is Empty then, set temp→ next = NULL and *p = head.

Step 4 - If it is Not Empty then, define a node pointer temp and initialize with head.

3 cases to be considered :

- Insert at position 1//Front insertion
- Insert at position n//shown below
- Insert in between//shown below

Step 5 - Keep moving the q to its next node until it reaches to the node after which we want to insert the temp (until q → data is equal to location, here location is the node value after which we want to insert the newNode).

Step 6 - Every time check whether q is reached to the last node or not. If it is reached to the last node then display **'Insertion not possible!!!**' and terminate the function. Otherwise move the q to the next node.

Step 7 - Finally, Set 'newNode → next = temp → next' and 'temp → next = newNode'

ADT - Deletion: Deletion operation in a Singly Linked List can be performed in 3 ways

1. Deleting from Beginning of the list :
    a. Node * Deletefront(struct node *temp);

2. Deleting from End of the list :
    a. Node * Deletednd(struct node *temp);

3. Deleting fromSpecific location in the list :
    a. Node * Deletepos(struct node *temp);

1 DELETING FROM THE BEGINNING OF THE LIST

Step 1 - Check whether list is Empty (***p == NULL**)

Step 2 - If it is Empty then, **return 0** and terminate

Step 3 - If it is Not Empty then, define a Node pointer **'q' and initialize it with p.**

Step 4 - Check whether list is having only one nod**e (q→ next == NULL)**

Step 5 - If it is TRUE then set **p = NULL and delete q** //Empty list Condition

Step 6 - If it is FALSE then set **p = q → next, and delete q.**

2. DELETING FROM THE END OF THE LIST

Step 1 - Check whether list is **Empty** (**p** == **NULL**)

Step 2 - If it is **Empty** then, return and terminate the function.

Step 3 - If it is **Not Empty** then, define two Node pointers **'prev'** and '**temp'** and initialize '**prev**' with **p.**

Step 4 - Check whether list has only one Node (**prev → next** == **NULL**)

Step 5 - If it is **TRUE**. Then, set **p** = **NULL** and delete **prev**. And terminate the function. //List is empty

Step 6 - If it is **FALSE**. Then, set '**temp = prev** ' and move **prev** to its next node. Repeat the same until it reaches the last node in the list. (until **prev → next** == **NULL**)

Step 7 - Finally, Set **temp→ next** = **NULL** and delete **temp1**

3. DELETION AT A SPECIFIC POSITION

Step 1 - Check whether list is Empty **(p == NULL)**

Step 2 - If it is Empty then, return and terminate

Step 3 - If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize **'temp1'=p;.**

Step 4 - Keep moving the temp1 until it reaches to the exact node to be deleted or to the last node. And every time set **'temp2 = temp1'** before moving the **'temp1' to its next node.**

Step 5 - If it is reached to the last node then display **"Deletion not possible!!!'. / or return 0 and terminate.**

Step 6 - If it is reached to the exact node which we want to delete, then check whether list is having only one node or not

Step 7 - If list has only one node and that is the node to be deleted, then set **p= NULL** and **delete temp1 (free(temp1)).**

Step 8 - If the list contains multiple nodes, then check whether temp1 is the first node in the list **(temp1 == head).**

Step 9 - If temp1 is the first node then move the p to the next node **(p = p→ next) and delete temp1.**

Step 10 - If temp1 is not the first node then check whether it is the last node in the list **(temp1 → next == NULL).**

Step 11 - If temp1 is the last node then set **temp2 → next = NULL** and **delete temp1 (free(temp1)).**

Step 12 - If temp1 is not the first node and not the last node then set **temp2 → next = temp1 → next and delete temp1 (free(temp1)).**

DISPLAY THE CONTENTS OF THE LIST

Step 1 - Check whether list is Empty **(p == NULL)**

Step 2 - If it is Empty then, **return 0 and terminate**

Step 3 - If it is Not Empty then, define a Node pointer **'q' and initialize it with a head.**

// Traverse using while or for to reach end of list

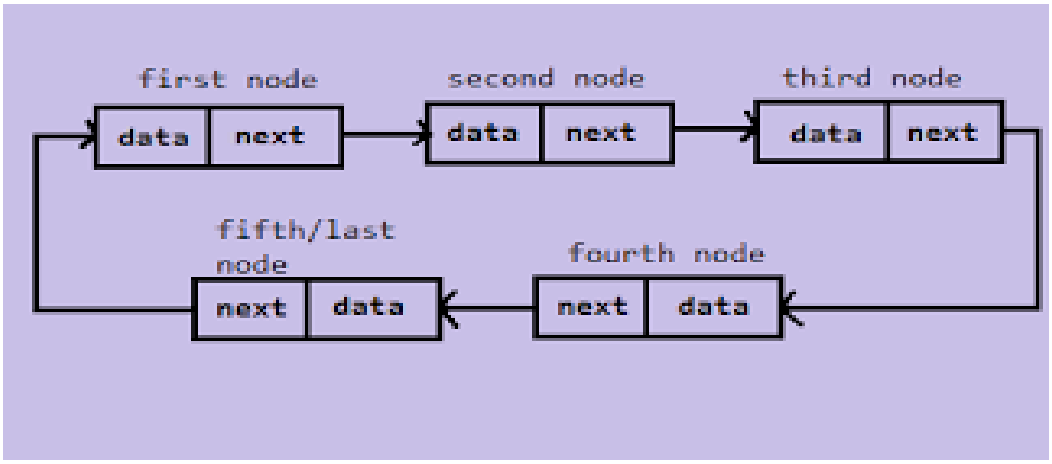
Step 4 - Keep displaying **q → data** with an arrow (--->) until q reaches to the last node

Step 5 - Finally display **q→ data** with an arrow pointing to NULL (q → data ---> NULL).

CIRCULAR LIST

- A type of linked list where all nodes are connected linearly in the form of a circle.
- It can be implemented using a singly linked list or doubly linked list.

- There is no null node unlike the singly linked list.
- The only difference between singly linked list and circular list is that, in a circular list the last node will point to the head rather than pointing to null.
- Pictorial representation of a circular list is as shown below:



In the above figure, the linked list contains five nodes. It can be observed that the last node(fifth node) points to the first node. Or in other words, the fifth node contains the address of the first node.

ADT of Circular List - BASIC OPERATIONS THAT CAN BE PERFORMED ON A CIRCULAR LIST

1. Insert
2. Delete
3. Display

INSERTION: 3 types of insertion operations on a singly linked list

1. Insertion at the beginning of the list
    a. Node* insertbeginning(struct node *temp, int data);

2. Insertion at the end of the list
    a. Node* endbeginning(struct node *temp, int data);

3. Insertion at a specific position
    a. Node* position_insert(struct node *temp, int data);

1. INSERTION AT THE BEGINNING OF THE CIRCULAR LIST

Step 1 - Create a new node as temp with given value.

Step 2 - Check whether list is Empty (p == NULL)

Step 3 - If it is Empty then, set p = temp and temp→next = p .

Step 4 - If it is Not Empty then, define a Node pointer 'q' and initialize with 'p''.

Step 5 - Keep moving the 'q' to its next node until it reaches the last node (until 'q → next == p').

Step 6 - Set temp→ next =p', 'p=temp' and 'q → next = p'

2. INSERTION AT THE END OF THE CIRCULAR LIST

Step 1 - Create a newNode with given value.

Step 2 - Check whether list is Empty (p== NULL).

Step 3 - If it is Empty then, set p = temp and temp → next = p.

Step 4 - If it is Not Empty then, define a node pointer q and initialize with head(q=p) .

Step 5 - Keep moving the q to its next node until it reaches to the last node in the list (until q → next ==p).//instead q->next =NULL

Step 6 - Set q → next = temp and temp→ next = p.

Deletion : Deletion operation can be performed in 3 ways

1. Delete from the beginning of the list
    a. Node * Deletefront(struct node *temp);

2. Delete from the end of the lis
    a. Node * Deletefront(struct node *temp);

3. Delete from a specific position

   Node * Deletefront(struct node *temp);

1. DELETION AT THE BEGINNING OF THE LIST

Step 1 - Check whether list is Empty p == NULL)

Step 2 - If it is Empty then, display 'List is Empty!!! Or return 0 and terminate the function.

Step 3 - If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize both 'temp1' and 'temp2' with p.(temp1=p, temp2=p;)

Step 4 - Check whether list is having only one node (temp1 → next == p)

Step 5 - If it is TRUE then set p = NULL and delete temp1 (Setting Empty list conditions)

Step 6 - If it is FALSE move the temp1 until it reaches to the last node. (until temp1 → next == p)

Step 7 - Then set p = temp2 → next, temp1 → next = p and delete temp2.

2. DELETION AT THE END OF THE LIST

Step 1 - Check whether list is **Empty** (**p** == **NULL**)

Step 2 - If it is **Empty** then, display **'List is Empty' or return 0 and** and terminate t

Step 3 - If it is **Not Empty** then, define two Node pointers **'temp1'** and '**temp2'** and initialize '**temp1**' with **p**.

Step 4 - Check whether list has only one Node (**temp1 → next** == **p**)

Step 5 - If it is **TRUE**. Then, set p = **NULL** and delete **temp1**. And terminate from the function. (Setting **Empty** list condition)

Step 6 - If it is **FALSE**. Then, set '**temp2 = temp1** ' and move **temp1** to its next node. Repeat the same until **temp1** reaches the last node in the list. (until **temp1 → next** == **p**)

Step 7 - Set **temp2 → next** = **p** and delete **temp1**

DOUBLY LINKED LIST

A Doubly linked list is a type of linked list in which traversal and access takes place in both the directions.

Node in a doubly linked list contains 3 fields:

1. Data Field
2. Left Link - which holds the address of the preceding node.
3. Right Link - which holds the address of the succeeding node.

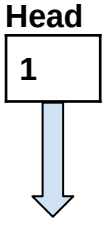
Dnode Structure definition of a Doubly Linked List is as follows

```
Struct Dnode
{
        Int data;
        Struct Dnode *prev;
        Struct Dnode *next;
}
```

Memory Representation of a doubly linked list

A doubly linked list consumes more space for every node.

But it's easy to manipulate the elements of the list since the list points both directions.



Data	left	Right
13	-1	4
15	1	6
19	4	8
57	6	-1

- First element of the list that is i.e. 13 stored at address 1.
- The head pointer points to the starting address 1.
- Since this is the first element being added to the list therefore the **prev** of the list **contains** null.
- The next node of the list resides at address 4 therefore the first node contains 4 in its next pointer.
- The list can be traversed in this way until it reaches the last Node pointing to NULL or -1.

ADT - BASIC OPERATIONS PERFORMED ON DOUBLY LINKED LIST

- Insertion.
- Deletion.

INSERTION : Insertion can be done in 3 ways

1. Insertion at the beginning of list
    a. Struct node *frontinsert(Struct node*temp, int value);

2. Insertion at the end of the list
    a. Struct node *frontend(Struct node*temp, int value);

3. Insertion at a specific position
    a. Struct node *frontpos(Struct node*temp, int value);

1. INSERTION AT THE BEGINNING OF THE LIST

Step 1 - Create a new node as **temp** , allocate memory dynamically and assign with value and set **temp → prev and temp->next as NULL.**

Step 2 - Check whether list is Empty **(p == NULL)**

Step 3 - If it is Empty then, assign NULL to **temp→ next and temp to head.**

Step 4 - If it is not Empty then, assign p to **temp → next and temp**.

2. INSERTION AT THE END OF THE DOUBLY LINKED LIST

Steps 1 to 3 : same as above

Step 4 - If it is not Empty, then, define a node pointer q and initialize with p.

Step 5 - Keep moving the q to its next node until it reaches to the last node in the list (until tq → next is equal to NULL).

Step 6 - temp to q→ next and temp to temp → previous

3. INSERTION AT A SPECIFIC POSITION OF THE DOUBLY LINKED LIST

Step 1 - Create a new node as **temp** , allocate memory dynamically and assign with a value.

Step 2 - Check whether list is Empty **(p == NULL)**

Step 3 - If it is Empty then, assign **NULL to both temp → previous & temp →next and set temp to p.**

Step 4 - If it is not Empty then, define two node pointers **q & temp2** and initialize **q with p.**

Step 5 - Keep moving the **q** to its next node until it reaches to the node after which we want to insert the temp (until **q → data is equal to location,** here location is the node value **after** which we want to insert the temp).

Step 6 - Each time keep checking whether **q** is reached to the last node. If it is reached to the last node then display **Insertion not possible and terminate the function**. Otherwise move the **q to the next node.**

Step 7 - Assign q → next to temp2, newNode to temp1 → next, q to temp → previous, temp2 to temp → next and temp to temp2 → previous

//Note : Can avoid usage of temp2 by making **q->previous->next=p and continue accordingly so on…**

DELETION OPERATION PERFORMED ON A DOUBLY LINKED LIST

Deletion of a node from a Doubly Linked List can be performed in 3 ways:

1. Deletion from the front of the list
    a. Struct node *deletefront(struct node*temp, struct node *prev, struct node *next);

2. Deletion from the end of the list
    a. Struct node *deleteend(struct node*temp, struct node *prev, struct node *next);

3. Deletion from a specific position
    a. Struct node *deletefront(struct node*temp, struct node *prev, struct node *next, int pos)

1. DELETION AT THE FRONT OF THE LIST

Step 1 - Check whether list is Empty **(p == NULL)**

Step 2 - If it is Empty then, display '**List is Empty**' and return 0 / terminate the function.

Step 3 - If it is not Empty then, define a Node pointer **'q'** and initialize it with **p**

Step 4 - Check whether list is having only one node **(q → previous is equal to q → next)**

Step 5 - If it is TRUE, then set **p** to NULL and delete q **(free(q))**

Step 6 - If it is FALSE, then assign **q → next to p**, p→ previous to head and **delete q. free(q).**

2. DELETION AT THE END OF THE LIST

Step 1 - Check whether list is Empty **(p == NULL)**

Step 2 - If it is Empty, then display **'List is Empty'** and return 0 / terminate the function.

Step 3 - If it is not Empty then, define a Node pointer **'q'** and initialize it with a p

Step 4 - Check whether list has only one Node (**q→ previous and q→ next = NULL)**

Step 5 - If it is TRUE, then assign **p to NULL and delete q.** And terminate from the function // empty list

Step 6 - If it is FALSE, then keep moving temp until it reaches to the last node in the list. (until **q → next = NULL**)

Step 7 - Assign temp → previous → next to null and delete q. **(free q).**

3. DELETION AT A SPECIFIC POSITION

Step 1 - Check whether list is Empty **(p == NULL)**

Step 2 - If it is Empty then, display **'List is Empty'** or return 0 and terminate the function

Step 3 - If it is not Empty, then define a Node pointer **'q'** and initialize it with p

Step 4 - Keep moving the temp until it reaches to the exact node to be deleted or to the last node.

Step 5 - If it is reached to the **last node**, then display **'Not Found'** and terminate the function.

Step 6 - If it is reached to the same node which is to be deleted, then check whether list is having only one node or not

Step 7 - If list has only one node and that is the node which is to be deleted then set **p to NULL and delete q (free(q)).//Only 1 node)**

Step 8 - If the list contains more than 1 node, then check whether temp is the first node in the list **(q == head)**.

Step 9 - If q is the first node, then move the head to the next node (q = q → next), set p of previous to NULL **(p → previous = NULL)** and delete f**ree(q)** - (Front deletion).

Step 10 - If temp is not the first node, then check whether it is the last node in the list **(q→ next == NULL).**

Step 11 - If temp is the last node then set q of previous to next to NULL **(q → previous → next = NULL)** and delete q **(free(q))**.//End deletion

Step 12 - If temp is not the first node and not the last node, then set q of previous of next to q of next **(q → previous → next = q → next),** q of next of previous to q of previous **(q → next → previous = q → previous)** and **delete q (free(q)).**//Somewhere in between.

MULTI LIST

A Multilist is a variation of Doubly Linked which is described as follows:

- each node has just 2 pointers
- the pointers are exact inverses of each other

In other words, In a Multi-linked list each node can have any number of pointers to other nodes, and there may or may not be inverses for each pointer.

Structure definitation of a multilist is as follows:

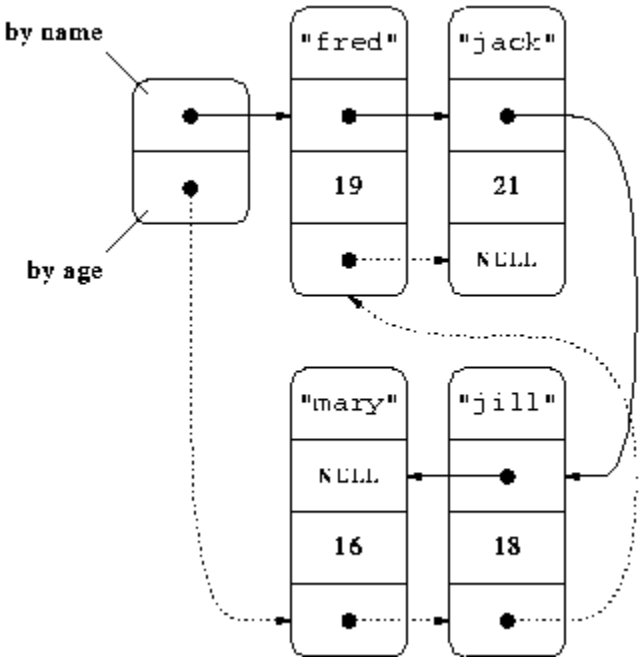
```
struct col_node {
        int col;
        int data;
        struct col_node *next_col;
};
struct row_node {
        int row;
        struct col_node *next_col;
        struct row_node *next_row;
};
```

Example 1: Multiorder of One set of Elements

The standard use of multi-linked lists is to organize a collection of elements in two different ways. For example, suppose my elements include the name of a person and his/her age.

E.g. (FRED, 19) (MARY,16) (JACK,21) (JILL,18)

The elements are ordered alphabetically and also by age. @ pointers are required : NEXT - alphabetically and NEXT - Age. List header will have 2 pointers one based on name, one based on age.



Example 2 : Sparse Matrix

- A sparse matrix is a matrix of numbers, as in mathematics, in which almost all the entries are zero.
- These arise frequently in engineering applications. the use of a normal Pascal array to store a sparse matrix is extremely wasteful of space - in an NxN sparse matrix typically only about N elements are non-zero.

Case Study : Design of Text Editor using SLL/DLL

Text editors are software programs that enable the user to create and edit text files. Notepad, Wordpad are some of the common editors used on Windows OS and vi, emacs, Jed, pico are the editors on UNIX OS.

Typical Features

- **Find and replace** – Text editors provide extensive facilities for searching and replacing text, either on groups of files or interactively. Advanced editors can use regular expressions to search and edit text or code.
- **Cut, copy, and paste** – most text editors provide methods to duplicate and move text within the file, or between files.
- Ability to handle UTF-8 encoded text.
- **Text formatting** – Text editors often provide basic visual formatting features like line wrap, auto-indentation, bullet list formatting using ASCII characters, comment

formatting, syntax highlighting and so on. These are typically only for display and do not insert formatting codes into the file itself.

- **Undo and redo** – As with word processors, text editors provide a way to undo and redo the last edit, or more. Often—especially with older text editors—there is only one level of edit history remembered and successively issuing the undo command will only "toggle" the last change. Modern or more complex editors usually provide a multiple-level history such that issuing the undo command repeatedly will revert the document to successively older edits. A separate redo command will cycle the edits "forward" toward the most recent changes. The number of changes remembered depends upon the editor and is often configurable by the user.

Develop a miniature text editing program which will allow a few simple commands, and is, therefore, quite primitive in comparison with a modern text editor or word processor.

Specifications:

Our text editor will allow us to read a file into memory, where we shall say that it is stored in a buffer. We shall consider each line of text to be a string, and the buffer will be a list of these lines. We shall then devise editing commands that will do **list operations** on the lines in the buffer and will do **string operations** on the characters in a single line.

Since, at any moment, the user either may be typing characters to be inserted into a line or may be giving commands, a text editor should always be written to be as forgiving of invalid input as possible, recognizing illegal commands, and asking for confirmation before taking any drastic action like deleting the entire buffer.

Here is a list of commands to be included in the text editor. Each command is given by typing the letter shown in response to the prompt '?'. The command letter may be typed in either uppercase or lowercase.

'**R**': Read the text file, whose name was given in the command line, into the buffer. Any previous contents of the buffer are lost. At the conclusion, the current line will be the first line of the file.

'**W**': Write the contents of the buffer to the text file whose name was given in the command line. Neither the current line nor the buffer is changed.

'**I**': Insert a single line typed in by the user at the current line number. The prompt 'I:' requests the new line.

'**D**': Delete the current line and move to the next line.

‘**F**’: Find the first line, starting with the current line that contains a target string that will be requested from the user.

‘**L**’: Show the length in characters of the current line and the length in lines of the buffer.

‘**C**’: Change the string requested from the user to a replacement text, also requested from the user, working within the current line only.

‘**Q**’: Quit the editor; terminate immediately.

‘**H**’: Print out help messages explaining all the commands. The program will also accept ‘?’ as an alternative to ‘H’.

‘**N**’: Next line: advance one line through the buffer.

‘**P**’: Previous line: back up one line in the buffer.

‘**B**’: Beginning: go to the first line of the buffer.

‘**E**’: End: go to the last line of the buffer.

‘**G**’: Go to a user-specified line number in the buffer.

‘**S**’: Substitute a line typed in by the user for the current line. The function should ask for the line number to be changed, print out the line for verification, and then request the new line.

‘**V**’: View the entire contents of the buffer, print out to the terminal.

Case Study: Design of a Symbol Table in an Assembler

In computer science, a symbol table is a data structure used by a language translator such as an assembler, compiler or interpreter, where each identifier (a.k.a. symbol) in a program's source code is associated with information relating to its declaration or appearance in the source. In other words, the entries of a symbol table store the information related to the entry's corresponding symbol.

The minimum information contained in a symbol table used by a translator includes the symbol's name, and its location or address.

The organization of the symbol table is the key to fast assembly. Even when working on a small program, the assembler may use the symbol table hundreds of times and, consequently, an efficient implementation of the table can cut the assembly time significantly even for short programs.

The symbol table is a dynamic structure. It starts empty and should support two operations, **insertion** and **search**. In a two-pass assembler, insertions are done only in

the first pass and searches, only in the second. In a one-pass assembler, both insertions and searches occur in the single pass. The symbol table does not have to support deletions, and this fact affects the choice of data structure for implementing the table. A symbol table can be implemented in many different ways but the following methods are almost always used:

- A linear array
- A sorted array with binary search
- Buckets with linked lists
- A binary search tree
- A hash table

A Linear Array

The symbols are stored in the first N consecutive entries of an array, and a new symbol is inserted into the table by storing it in the first available entry (entry N + 1) of the array. The variable N is initially set to zero, and it always points to the last entry in the array. An insertion is done by:
Testing to make sure that N < lim (the symbol table is not full). Incrementing N by 1. Inserting the name, value (location) , and any other fields, using N as an index.

The insertion takes fixed time, independent of the number of symbols in the table. To search, the array of names is scanned entry by entry. The number of steps involved varies from a minimum of 1 to a maximum of N. Every search for a non-existent symbol involves N steps, thus a program with many undefined symbols will be slow to assemble because the average search time will be high. Assuming a program with only a few undefined symbols, the average search time is N/2. In a two-pass assembler, insertions are only done in the first pass so, at the end of that pass, N is fixed. All searches in the second pass are performed in a fixed table. In a one-pass assembler, N grows during the pass, and thus each search takes an average of N/2 steps, but the values of N are different.

Advantages: Fast insertion. Simple operations.
Disadvantages: Slow search, specially for large values of N. Fixed size

A Sorted Array

The same as a linear array, but the array (actually two arrays or more, for the name, value (location), and any other attributes) is sorted, by name, after the first pass is completed. This, of course, can only be done in a two-pass assembler. To find a symbol in such a table, binary search is used, which takes an average of $\log_2 N$ steps. The difference between N and $\log_2 N$ is small when N is small but, for large values of N, the difference can get large enough to justify the additional time spent on sorting the table.

Advantages: Fast insertion and fast search.
Disadvantages: The sort takes time, which makes this method useful only for a large number of symbols (at least a few hundred).

Buckets with Linked Lists

An array of 26 entries is declared, to serve as the start of the buckets. Each entry points to a bucket that is a linked list of all those symbols that start with the same letter. Thus all the symbols that start with a 'C' are linked together in a list that can be reached by following the pointer in the third entry of the array. Initially all the buckets are empty (all pointers in the array are null). As symbols are inserted, each bucket is kept sorted by symbol name. Notice that there is no need to actually sort the buckets. The buckets are kept in sorted order by carefully inserting each new symbol into its proper place in the bucket. When a new symbol is presented, to be inserted in a bucket, the bucket is first located by using the first character in the symbol's name (one step). The symbol is then compared to the first symbol in the bucket (the symbol names are compared). If the new symbol is less (in lexicographic order) than the first, the new one becomes the first in the bucket. Otherwise, the new symbol is compared to the second symbol in the bucket, and so on. Assuming an even distribution of names over the alphabet, each bucket contains an average of N/26 symbols, and the average insertion time is thus 1+(N/26)/2=1+ N/52. For a typical program with a few hundred symbols, the average insertion requires just a few steps.

A search is done by first locating the bucket (one step), and then performing the same comparisons as in the insertion process above. The average search thus also takes 1 + N/52 steps.

Such a symbol table has a variable size. More nodes can be allocated and added to the buckets, and the table can, in principle, use the entire available memory.

Advantages: Fast operations. Flexible table size.
Disadvantages: Although the number of steps is small, each step involves the use of a pointer and is therefore slower than a step in the previous methods (that use arrays). Also, some programmers always tend to assign names that start with an A. In such a case all the symbols will go into the first bucket, and the table will behave essentially as a linear array.

Such an implementation is recommended only if the assembler is designed to assemble large programs, and the operating system makes it convenient to allocate storage for list nodes.

A Binary Search Tree

This is a general data structure used not just for symbol tables, and is quite efficient. It can be used by either a one pass or two pass assembler with the same efficiency.

The table starts as an empty binary tree, and the first symbol inserted into the table becomes the root of the tree. Every subsequent symbol is inserted into the table by (lexicographically) comparing it with the root. If the new symbol is less than the root, the program moves to the left son of the root and compares the new symbol with that son. If the new symbol is greater than the root, the program moves to the right son of the root and compares as above. If the new symbol turns out to be equal to any of the existing tree nodes, then it is a doubly-defined symbol. Otherwise, the comparisons continue until a

node is reached that does not have a son. The new symbol becomes the (left or right) son of that node.

Example: Assuming that the following symbols are defined, in this order, in a program.

BGH, J12, MED, CC, ON, TOM, A345, ZIP, QUE, PETS

Symbol BGH becomes the root of the tree, and the final binary search tree is shown in below figure.

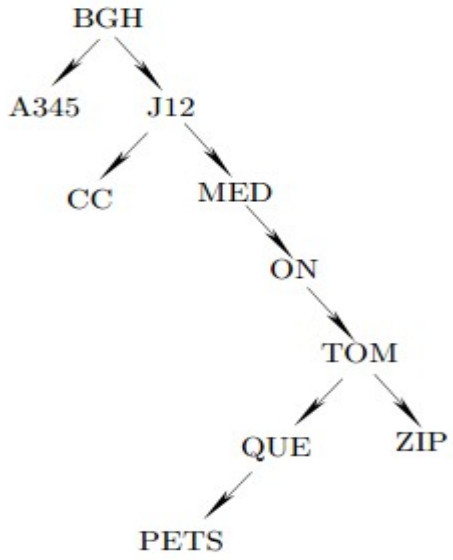


Figure: Binary Search Tree

The minimum number of steps for insertion or search is obviously 1. The maximum number of steps depends on the height of the tree. The tree in the above figure has a height of 7, so the next insertion will require from 1 to 7 steps. The height of a binary tree with N nodes varies between $\log_2 N$ (which is the height of a fully balanced tree), and N (the height of a skewed tree). It can be proved that an average binary tree is closer to a balanced tree than to a skewed tree, and this implies that the average time for insertion or search in a binary search tree is of the order of $\log_2 N$.

Advantages: Efficient operation (as measured by the average number of steps). Flexible size.
Disadvantages: Each step is more complex than in an array-based symbol table.

The recommendations for use are the same as for the previous method.

A Hash Table

This method comes in two varieties, closed hash, which is a fixed-size array and open hash, which uses pointers and has a variable size.

Closed hashing

A closed hash table is an array (actually two or more arrays, for the name, value (location), and any other attributes), normally of size 2^N , where each symbol is stored in an entry. To insert a new symbol, it is necessary to obtain an index to the entry where the symbol will be stored. This is done by performing an operation on the name of the symbol, an operation that results in an N-bit number. An N-bit number has a value between 0 and $2^N - 1$ and can thus serve as an index to the array. The operation is called hashing and is done by hashing, or scrambling, the bits that constitute the name of the symbol. For example, consider 6-character names, such as abcdef. Each character is stored in memory as an 8-bit ASCII code. The name is divided into three groups of two characters (16-bits) each, ab cd ef. The three groups are added, producing an 18-bit sum. The sum is split into two 9-bit halves which are then multiplied to give an 18-bit product. Finally N bits are extracted from the middle of the product to serve as the hash index. The hashing operations are meaningless since they operate on codes of characters, not on numbers. However, they produce an N-bit number that depends on all the bits of the original name.

A good hash function should have the following two properties:

- It should consider all the bits in the original name. Thus when two names that are slightly different are hashed, there should be a good chance of producing different hash indexes.
- For a group of names that are uniformly distributed over the alphabet, the function should produce indexes uniformly distributed over the range 0 ... $2^N - 1$.

Once the hash index is produced, it is used to insert the symbol into the array. Searching for symbols is done in an identical way. The given name is hashed, and the hashed index is used to retrieve the value (location) and any other attributes from the array.

Ideally, a hash table requires fixed time for insert and search, and can be an excellent choice for a large symbol table. There are, however, two problems associated with this method namely, collisions and overflow, that make hash tables less than ideal.

Collisions involve the case where two entirely different symbol names are hashed into identical indices. Names such as SYMB and ZWYG6 can be hashed into the same value, say, 54. If SYMB is encountered first in the program, it will be inserted into entry 54 of the hash table. When ZWYG6 is found, it will be hashed, and the assembler should discover that entry 54 is already taken. The collision problem cannot be avoided just by designing a better hash function. The problem stems from the fact that the set of all possible symbols is very large, but any given program uses a small part of it. Typically, symbol names start with a letter, and consist of letters and digits only. If such a name is limited to six characters, then there are 26×365 ($\approx$

1.572 billion) possible names. A typical program rarely contains more than, say, 500 names, and a hash table of size 512 (= 29) may be sufficient. When 1.572 billion names are mapped into 512 positions, more than 3 million names will map into each position. Thus even the best hash function will generate the same index for many different names, and a good solution to the collision problem is the key to an efficient hash table.

The simplest solution involves a linear search. All entries in the symbol table are originally marked as vacant. When the symbol SYMB is inserted into entry 54, that entry is marked occupied. If symbol ZWYG6 should be inserted into entry 54 and that entry is occupied, the assembler tries entries 55, 56 and so on. This implies that, in the case of a collision, the hash table degrades to a linear table.

Another solution involves trying entry 54 + P where P and the table size are relative primes. In either case, the assembler tries until a vacant entry is found or until the entire table is searched and found to be all occupied.

It is seen that when the hash table gets more than 50%–60% full, performance suffers, no matter how good the hashing function is. Thus a good hash table design makes sure that the table never gets more than 60% occupied. At that point the table is considered overflowed.

The problem of hash table overflow can be handled in a number of ways. Traditionally, a new, larger table is opened and the original table is moved to the new one by rehashing each element. The space taken by the original table is then released. A better solution, though, is to use open hashing.

Open hashing

An open hash table is a structure consisting of buckets, each of which is the start of a linked list of symbols. It is very similar to the buckets with linked lists discussed above. The principle of open hashing is to hash the name of the symbol and use the hash index to select a bucket. This is better than using the first character in the name, since a good hash function can evenly distribute the names over the buckets, even in cases where many symbols start with the same letter.

OTHER OPERATION PERFORMED ON ANY LIST:

1. Sort the List
2. Addition of Two List
3. Display List in the reverse order
4. Display every alternate nodes in the list
5. Find the length of the list
6. Search for an element in the list.
7. Swap references without swapping the data
8. Remove the duplicates elements in list
9. Separate even and odd nodes in the list
10. Check for palindrome

APPLICATIONS OF LINKED LIST

1. Implementation of other Data Structures like - Stacks, Queues, Trees, Graphs, Hash Tables
2. Allocation and deallocation of memory
3. Performing arithmetic operations on long integers - Addition, Subtraction, Multiplication and Division
4. Representation of Sparse Matrix
5. Maintaining a Dictionary
6. Designing a Text Editor, Photo editor
7. Music Player.
8. Blockchain(Bitcoin)
9. Pattern Matching
10. Database applications - Student / employee record details, hotel management, reservation system etc…

REFERENCES

1. "Data Structures and Program Design in C", Robert Kruse, Cl Tondo, Pearson Education, 2nd Edition, 2007.
2. https://en.wikipedia.org/wiki/Data_structure#:~:text=In%20computer%20science%2C%20a%20data,be%20applied%20to%20the%20data.
3. https://www.programiz.com/c-programming/c-dynamic-memory-allocation.
4. http://staff.um.edu.mt/csta1/courses/lectures/csa2060/c8a.html
5. https://computer.howstuffworks.com/c-programming11.htm.
6. https://www.careerride.com/c-static-memory-dynamic-memory-allocation.aspx
7. https://www.geeksforgeeks.org/data-structures/
8. https://www.geeksforgeeks.org/editors-types-system-programming/
9. https://en.wikipedia.org/wiki/Text_editor
10. https://en.wikipedia.org/wiki/Symbol_table
11. https://www.davidsalomon.name/assem.advertis/asl.pdf

Write a C Program to implement a Singly Linked List with the following Operations:

1. **Insert at the beginning of the list**
2. **Insert at the end of the List**
3. **Delete at the beginning of the List**
4. **Delete at the end of the list**
5. **Insert at a specific position**
6. **Delete at a specific position**
7. **Reverse the list**
8. **Display the contents of the list**
9. **Search for a node.**

```
#include<stdio.h>
#include<stdlib.h>
//define structure
struct node
{
  int data;
  struct node* next;
};
//driver program
int main()
{
  struct node* first;
  int ch,x,y;
  void insert_tail(struct node **, int);
  void insert_head(struct node **, int);
  void display(struct node*);
  void delete_node(struct node **,int);
  void delete_pos(struct node **,int);
  void reverse(struct node **);
  void insert_pos(struct node **, int,int);
  first=NULL; // points to the first node
  while(1)
  {
    display(first);
    printf("\n1..Insert tail\n");
    printf("2..Insert head\n");
    printf("3..Display\n");
    printf("4..delete node\n");
```

```
    printf("5..delete position\n");
    printf("6..reverse list..\n");
    printf("7..insert position\n");
    printf("8..Exit\n");
    scanf("%d",&ch);
    switch(ch)
    {
      case 1:printf("Enter the number\n");
             scanf("%d",&x);
             insert_tail(&first,x);
             break;
      case 2:printf("Enter the number\n");
             scanf("%d",&x);
             insert_head(&first,x);
             break;
      case 3: display(first);
             break;
      case 4:printf("Enter the value of node to be deleted\n");
             scanf("%d",&x);
             delete_node(&first,x);
             break;
      case 5:printf("Enter the position of node to be deleted\n");
             scanf("%d",&x);
             delete_pos(&first,x);
             break;
       case 6:reverse(&first);
              break;
       case 7:printf("Enter the value & position \n");
             scanf("%d %d",&x,&y);
             insert_pos(&first,x,y);
             break;
    }
  }
}
```

//insert at a given position

```
void insert_pos(struct node **p , int x,int pos)
```

```
{
  struct node *prev, *temp, *q;
  int i;
  //create node
  temp=(struct node*)malloc(sizeof(struct node));
  temp->data=x;
  temp->next=NULL;

   i=1;
  q=*p;
  prev=NULL;

  //go to the desired position
  while((q!=NULL)&&(i<pos))
 {
   prev=q;
   q=q->next;
   i++;
 }
 if(q!=NULL)//position found
 {
    if(prev==NULL)//check if it is first position
      {
       temp->next=*p;//insert at the first position
       *p=temp;
      }
     else// between 2 and the last position
      {
         prev->next=temp;
         temp->next=q;
      }
  }
  else//q==NULL
  {
    if(prev==NULL)//empty list, insert the first node
      *p=temp;
    else if(i==pos)//insert at the end
          prev->next=temp;
     else
        printf("Invalid position..\n");
  }
```

```
}
```

//function to reverse the list

```
void reverse(struct node **p)
{
   struct node *curr, *prev, *temp;

   prev=NULL;
   curr=*p;

   while(curr!=NULL)
   {
    temp=curr->next;
    curr->next=prev;
    prev=curr;
    curr=temp;
   }
   *p=prev;
 }
```

//delete a node at a given position

```
void delete_pos(struct node **p, int pos)
{
  int i;
  struct node *q,*prev;
  prev=NULL;

  i=1;
   q=*p;

 //move forward till the position is found or end of list is reached
  while((q!=NULL)&&(i<pos))
  {
    prev=q;
    q=q->next;
```

```
      i++;
    }
   if(q==NULL) // end of list reached
      printf("invalid position.\n");
     else if(prev==NULL)//first node is being deleted
      *p=q->next;//make second as the first node
     else
       prev->next=q->next;
     free(q);
  }


 //delete a node given its value.
//deletes only the first occurrence of the node
  void delete_node(struct node **p, int x)
  {
    struct node *q,*prev;
    prev=NULL;

    //keep moving forward till the node to be deleted
    //is found or you go beyond the last node
    q=*p;
    while((q!=NULL)&&(q->data!=x))
    {
      prev=q;
      q=q->next;
     }
     if(q==NULL)
      printf("the node not found..\n");
     else if(prev==NULL)//first node is being deleted
      *p=q->next;//make second as the first node
     else
       prev->next=q->next;
     free(q);
    }



//insert the node at the front of the list
  void insert_head(struct node **p,int x)
  {
```

```
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
     temp->next=NULL;

    //check if this is the first element
    if(*p==NULL)
      *p=temp;
    else
     {
        temp->next=*p;
        *p=temp;
     }
}

//insert the node at the end of the list
  void insert_tail(struct node **p,int x)
  {
     struct node *temp,*q;
   //create node
     temp=(struct node*)malloc(sizeof(struct node));
   //copy the values in the node

    temp->data=x;
    temp->next=NULL;

    //check if this is the first node
    if(*p==NULL)//checking the content of first
      *p=temp;
    else
      {
         //go to the end of the list
       q=*p;
       while(q->next!=NULL)
          q=q->next;//move forward
       q->next=temp;//link the new node to the last node
     }
  }

 //displays the list
```

```
void display(struct node *p)
{
  if(p==NULL)
   printf("Empty List..\n");
  else
   {
     while(p!=NULL)
     {
       printf("%d ->",p->data);
       p=p->next;
     }
   }
}
```

//delete the first node

```
void delete_first(struct node **p)
{
  struct node *q;
  //check for empty list
   if(*p==NULL)
        printf(“Empty List..”);
   else
     {
        q=*p;
       *p= q->next;
         free(q);
    }
}
```

//delete the last node

```
void delete_last(struct node **p)
{
  struct node *q,*prev;
     //check for empty list
  if(*p==NULL)
    printf(“Empty List..\n”);
 else
   {
     prev=NULL;
     q=*p;
```

```
     //go to the last node
      while( q->next!=NULL)
     {
        prev=q;
       q=q->next;
       }
    if(prev!=NULL) // check if there is only one node
        prev->next=NULL;
    free(q);
 }

//search for a key , using linear search
void search(struct node **p, int key)
{
   struct node *q;

   if(*p==NULL)
    printf("Empty List\n");
 else
    {
        q=*p;
        while((q!=NULL) &&(q->data!=key))
            q=q->next;
        if(q==NULL)
          printf("key found..\n")
else
     printf("key found\n")
  }
}
```

Program to create an ordered list, elements will be inserted in the ascending order

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
  int data;
  struct node* next;
};

void display(struct node*);
```

```
void insert_order(struct node**,int);
int main()
{
  struct node *first;
  int x;
   first=NULL; //points to the first node

  while(1)
  {
    printf("\nEnter the number..\n");
    scanf("%d",&x);
     if(x==0)
       break;
    insert_order(&first,x);
    display(first);
  }
}

 void display(struct node *p)
 {
   printf("The list..\n");
   while(p!=NULL)
   {
     printf("%d->",p->data);
     p=p->next;
   }
}

  void insert_order(struct node **p, int x)
  {
    struct node *temp, *prev, *q;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=NULL;

    q=*p;
    prev=NULL;;

    //move forward until the position of the element is found
    while((q!=NULL)&&(x>q->data))
```

```
  {
    prev=q;
    q=q->next;
   }
  if(q!=NULL)
  {
    if(prev==NULL)//inserting the smallest number, insert first
    {
      temp->next=q;
      *p=temp;
     }
   else// insert somewhere in middle of the list
    {
      temp->next=q;
      prev->next=temp;
    }
  }
 else//q==NULL
 {
   if(prev==NULL)//empty list, first node inserted
        *p=temp;
  else
    prev->next=temp;//largest no, insert at end
 }
}
```

Write a C Program to perform the following operations on a Doubly Linked List:

1. **Insert at the beginning of the list**
2. **Insert at the end of the list**
3. **Insert at a specific position**
4. **Delete at the beginning of the list**
5. **Delete at the end of the list.**
6. **Delete at a specific position in the list.**
7. **Display the contents of the list.**

```
#include<stdio.h>
#include<stdlib.h>
struct node
```

```
{
   int data;
   struct node *prev;
   struct node *next;
};
void insert_head(struct node**, int);
void insert_tail(struct node **, int);
void delete_node(struct node**, int);
void insert_pos(struct node **,int ,int);
void delete_pos(struct node**, int);
void display(struct node*);
void delete_last(struct node **);
void delete_first(struct node **);
int main()
{
  struct node *first;
  first=NULL;
  int x,ch,pos;
  while(1)
  {
    display(first);
    printf("\n1..Insert Head..\n");
    printf("2..Insert Tail..\n");
    printf("3..Delete First..\n");
    printf("4..Delete Last..\n");
    printf("5..Delete node..\n");
    printf("6..Delete at position\n");
    printf("7..Insert at position\n");
    scanf("%d",&ch);

    switch(ch)
    {
       case 1: printf("Enter the value..\n");
               scanf("%d",&x);
               insert_head(&first,x);
               break;
        case 2: printf("Enter the value..\n");
               scanf("%d",&x);
               insert_tail(&first,x);
               break;
        case 3:delete_first(&first);
```

```
            break;
        case 4:delete_last(&first);
            break;
        case 5: printf("Enter the value..\n");
            scanf("%d",&x);
            delete_node(&first,x);
            break;
       case 6: printf("Enter the position..\n");
            scanf("%d",&x);
            delete_pos(&first,x);
            break;
       case 7: printf("Enter the value and position..\n");
            scanf("%d %d",&x,&pos);
            insert_pos(&first,x,pos);
            break;

   }

 }
}
```

//delete the first node

```
void delete_first(struct node **p)
  {
    struct node *q;
    q=*p;

    if((q->prev==NULL)&&(q->next==NULL))//only one node
      *p=NULL;
    else //more than one node in the list
     {
        *p=q->next;
        (*p)->prev=NULL;
     }
    free(q);
   }
```

//delete the last node

```
 void delete_last(struct node **p)
 {
```

```
    struct node *q;
    q=*p;

    if((q->prev==NULL)&&(q->next==NULL))//only one node
       *p=NULL;
    else //more than one node in the list
     {
        while(q->next!=NULL)
          q=q->next;

      q->prev->next=NULL;
      }
    free(q);
  }
```

//insert at a given position

```
void insert_pos(struct node **p,int x,int pos)
{
  struct node *temp, *q;

 //create a node

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->prev=temp->next=NULL;

   q=*p;
   int i=1;

   //go to the position

   while((q->next!=NULL)&&(i<pos))
    {
      i++;
       q=q->next;
     }
   if(q->next!=NULL)//position found
   {
     //check if first position
      if(q->prev==NULL)
      {
```

```
          //insert in first position
          temp->next=q;
          q->prev=temp;
          *p=temp;
        }
       else
       {
        //insert somewhere in the middle of list
        //but not the last but one position
        q->prev->next=temp;
        temp->prev=q->prev;
        temp->next=q;
         q->prev=temp;
       }
    }
    else//q->next==NULL
     {
       if(i==pos)//insert at the last but one position
       {
         q->prev->next=temp;
        temp->prev=q->prev;
        temp->next=q;
         q->prev=temp;
     }
     else if(i==pos-1)//insert after the last node
     {
        q->next=temp;
         temp->prev=q;
     }
     else
       printf("Invalid position..\n");
    }
}
```

//delete at a given position

```
 void delete_pos(struct node**p, int pos)
   {
     struct node *q;

     //find the node to be deleted
     q=*p;
```

```
 int i=1;
 while((q!=NULL)&&(i<pos))
 {
  i++;
  q=q->next;
 }

 if(q!=NULL)//position found
 {
  if((q->prev==NULL)&&(q->next==NULL))//only one node
      *p=NULL;
  else if(q->prev==NULL)//first position
  {
     *p=q->next;
     (*p)->prev=NULL;
  }
   else if(q->next==NULL)//last position
      q->prev->next=NULL;
  else //somewhere in middle
   {
      q->prev->next=q->next;
       q->next->prev=q->prev;
   }
  free(q);
 }
 else//q=NULL
  printf("Invalid position.\n");
}
```

//delete the first occurrence of a node given its value

```
 void delete_node(struct node**p, int x)
 {
  struct node *q;

  //find the node to be deleted
  q=*p;

  while((q!=NULL)&&(q->data!=x))
   q=q->next;
```

```
  if(q!=NULL)//node found
  {
    if((q->prev==NULL)&&(q->next==NULL))//only one node
         *p=NULL;
    else if(q->prev==NULL)//first node
    {
       *p=q->next;
       (*p)->prev=NULL;
    }
     else if(q->next==NULL)//last node
        q->prev->next=NULL;
    else //somewhere in middle
      {
         q->prev->next=q->next;
          q->next->prev=q->prev;
      }
    free(q);
  }
  else//q=NULL
    printf("Node not found..\n");
}
```

//insert a node at the front of the list

```
 void insert_head(struct node **p, int x)
 {
    struct node *temp;
   //create a node

   temp=(struct node*)malloc(sizeof(struct node));
   temp->data=x;
   temp->prev=temp->next=NULL;

  //if this is the first node

   if(*p==NULL)
      *p=temp;//make first point to temp
    else
    {
      temp->next=*p;//link the new node to first node
```

```
        (*p)->prev=temp;
        *p=temp;//make  first point to new node
      }
   }
```

//display the contents of the list

```
void display(struct node *p)
 {
    if(p==NULL)
       printf("\nEmpty List..\n");
    else
     {
       while(p!=NULL)
       {
         printf("%d<->",p->data);
         p=p->next;
       }
     }
 }
```

//insert the node at the end of the list

```
void insert_tail(struct node **p, int x)
{
    struct node *temp,*q;
    //create a node

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->prev=temp->next=NULL;

   //if this is the first node

    if(*p==NULL)
        *p=temp;//make first point to temp
    //go to the end of the list
   else
   {
    q=*p;
    while(q->next!=NULL)
```

```
      q=q->next;

    q->next=temp;//link the new node to last node
    temp->prev=q;
   }
  }
```

Implement the following operations on a Circular Singly Linked List:

1. **Insert at the beginning of the list**
2. **Insert at the end of the list**
3. **Delete a node given its value**
4. **Display the list**

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
  int data;
  struct node* next;
};
void insert_head(struct node**,int);
void insert_tail(struct node**,int);
void delete_node(struct node**,int);
void display(struct node*);

int main()
{
  struct node *last;
  int ch,x,pos;
  last=NULL;//pointer to the last node of the list

  while(1)
  {
    display(last);
    printf("\n1..Insert Head\n");
    printf("2..Insert Tail\n");
    printf("3..Delete a Node..\n");
    printf("4..Display\n");
    printf("5..Exit\n");
```

```
    scanf("%d",&ch);
    switch(ch)
    {
     case 1:printf("Enter the number...");
            scanf("%d",&x);
            insert_head(&last,x);
            break;
     case 2:printf("Enter the number...");
            scanf("%d",&x);
            insert_tail(&last,x);
            break;
    case 3: printf("Enter the value of the node to be deleted...");
            scanf("%d",&x);
            delete_node(&last,x);
            break;

     case 4:display(last);
            break;
     case 5:exit(0);
   }
 }
}

//delete node given its value
  void delete_node(struct node**p,int x)
{
  struct node *prev,*q,*r;

  q=*p;//copy of the last node address
  prev=q;//keep track the previous node

  r=q->next;//first node

  //move forward till you find the data
  //or you stop at the last node
  while((r!=q)&&(r->data!=x))
  {
    prev=r;
    r=r->next;
  }
  if(r->data==x)//node found
```

```
 {
    if(r->next==r)//only one node
      *p=NULL;
    else
     {
        prev->next=r->next;
        if(r==q)//deleting the last node
          *p=prev;
      }
     free(r);
  }
  else
    printf("Node not found..\n");
}

//insert a node at the head of the list
 void insert_head(struct node**p,int x)
 {
   struct node *temp;

   temp=(struct node*)malloc(sizeof(struct node));
   temp->data=x;
   temp->next=temp;

   if(*p==NULL)
     *p=temp;
   else
    {
       temp->next=(*p)->next;
       (*p)->next=temp;
    }
 }

//display the contents of the list
void display(struct node *p)
{
   struct node *q;

  if(p==NULL)
   printf("\nEmpty list..\n");
 else
```

```
  {
  q=p->next;
   while(q!=p)
   {
      printf("%d ->",q->data);
      q=q->next;
   }
  printf("%d ->",q->data);//last node
  }
}

//insert a node at the end of the list
  void insert_tail(struct node**p,int x)
 {
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=temp;

    if(*p==NULL)
      *p=temp;
    else
     {
        temp->next=(*p)->next;
        (*p)->next=temp;
         *p=temp;
     }
  }
```

Implement the following operations on a Circular Singly Linked List:
The list has a header node. The header node keeps the count of the number of nodes in the list. Empty List contains only the header node

1. **Insert at the beginning of the list**
2. **Insert at the end of the list**
3. **Delete a node given its value**
4. **Display the list**

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
  int data;
  struct node *next;
};

void insert_head(struct node *,int);
void insert_tail(struct node *,int);
void display(struct node *);
struct node *create_head();
void delete_node(struct node*,int);

//implementing circular list with a header node
int main()
{
  struct node *head;
   int x,ch;
   head=create_head(); //head points to the header node
   while(1)
  {
    display(head);
    printf("\n1..Insert Head\n");
    printf("2..Insert Tail\n");
    printf("3..Delete a Node..\n");
    printf("4..Exit\n");
    scanf("%d",&ch);
    switch(ch)
    {
     case 1:printf("Enter the number...");
            scanf("%d",&x);
            insert_head(head,x);
            break;
     case 2:printf("Enter the number...");
```

```
            scanf("%d",&x);
            insert_tail(head,x);
            break;

      case 3: printf("Enter the value of the node to be deleted...");
            scanf("%d",&x);
            delete_node(head,x);
            break;

      case 4:exit(0);
    }
  }
}


//creates a header node

 struct node *create_head()
 {
   struct node *temp;
   temp=(struct node*)malloc(sizeof(struct node));
   temp->data=0;
   temp->next=temp;
   return temp;
  }

//insert a node at the front of the list
//i.e. after the header node
 void insert_head(struct node *p,int x)
 {
   struct node *temp;
   temp=(struct node*)malloc(sizeof(struct node));
   temp->data=x;
   temp->next=p->next; // insert after the header node
   p->next=temp;
   p->data++;//increment the count of the nodes
  }

 //display the contents of the list
 void display(struct node *p)
 {
```

```
  struct node *q;
  q=p;

  while(p->next!=q)
  {
   printf("%d-> ",p->data);
   p=p->next;
  }
  printf("%d ",p->data);
 }


//insert at the end of the list
 void insert_tail(struct node *p,int x)
 {
  struct node *temp,*q;
   temp=(struct node*)malloc(sizeof(struct node));
   temp->data=x;

   q=p->next;

   while(q->next!=p)
    q=q->next;

   temp->next=q->next;//or temp->next=p
   q->next=temp;
   p->data++;
 }



//delete a node given its value
void delete_node(struct node *p, int x)
 {
   struct node *prev,*q;

   q=p->next;
   prev=p;

  //move forward until the node to be deleted is found or the header node is reached
   while((q!=p)&&(q->data!=x))
```

```
   {
    prev=q;
    q=q->next;
   }
   if(q==p)
    printf("Node not found..\n");
  else
  {
   prev->next=q->next; //delete the node
   free(q);
   p->data--;//decrement the count in the header node
  }
}
```

Program to merge two singly linked list

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
  int data;
  struct node* next;
};

void display(struct node*);
void insert_order(struct node**,int);
void createlist(struct node**);
void merge(struct node*,struct node*,struct node**);
int main()
{
  struct node *first,*second,*third;
  first=NULL;
  second=NULL;
  third=NULL;
 printf("Creating the first List..\n");
 createlist(&first);
 display(first);
 printf("\nCreating the second List..\n");
 createlist(&second);
 display(second);
 printf("\nMerging the lists..\n");
```

```
 merge(first,second,&third);
 display(third);
}

void display(struct node *p)
 {
   printf("The list..\n");
   while(p!=NULL)
   {
     printf("%d->",p->data);
     p=p->next;
   }
}

void createlist(struct node**p)
{
  int x;
  while(1)
  {
    printf("\nEnter the number..\n");
    scanf("%d",&x);
     if(x==0)
       break;
    insert_order(p,x);
  }
}

void insert_order(struct node** p,int x)
 {
  struct node *temp,*prev,*q;

  temp=(struct node*)malloc(sizeof(struct node));
  temp->data=x;
  temp->next=NULL;

  q=*p;//copy address of the first node
  prev=NULL;

  while((q!=NULL)&&(x>q->data))
  {
    prev=q;
```

```
    q=q->next;
  }
  if(q!=NULL)//position found,x<q->data
  {
   if(prev==NULL)//first position, insert as the first node
    { temp->next=q;
    *p=temp;
    }
   else//insert somewhere in middle
   {
    prev->next=temp;
    temp->next=q;
   }
  }
  else//q==NULL,
  {
    if(prev==NULL)//empty list,insert the node as the first node
      *p=temp;
    else
     prev->next=temp;//insert at the end of the list
 }
}

void insert_tail(struct node **p, int x)
  {

   struct node *temp, *q;
    //create node
     temp=(struct node*)malloc(sizeof(struct node));
     temp->data = x;
     temp->next=NULL;

     //if list empty
     if(*p==NULL)
       *p=temp;
     else
     {
       q=*p;//copy the address of teh first node of the list
     //keep moving till you stop at the last node
      while(q->next!=NULL)
       q=q->next;
```

```
      q->next=temp;//link the last node to new node
     }
  }
```

//merging the two lists

```
void merge(struct node* p,struct node* q,struct node** t)
{
  while((p!=NULL)&&(q!=NULL))
  {
    if(p->data <= q->data)
    {
      insert_tail(t,p->data);
      p=p->next;
    }
   else
   {
     insert_tail(t,q->data);
      q=q->next;
   }
  }
  if(p==NULL)//end of the first list
  {
    while(q!=NULL)//copy all the elements of the second list
    {
      insert_tail(t,q->data);
      q=q->next;
    }
  }
  else// q==NULL end of the second list
  {
    while(p!=NULL)//copy all the elements of the first list
    {
      insert_tail(t,p->data);
      p=p->next;
     }
   }
}
```

Implementation of Sparse Matrix Using Multilist:
Representing a sparse matrix as a multi list

```
#include<stdio.h>
#include<stdlib.h>
//used to store non zero value
struct col_node {
        int col; //column index
        int data; //non zero value
        struct col_node *next_col;
};

//represents the row
struct row_node {
        int row;
        struct col_node *next_col; // points to the first non zero value of that row
        struct row_node *next_row; //points to the next row
};
struct row_node *create_rows(int);
void insert_list(struct row_node*, int,int,int);
void display(struct row_node*);
int main()
{
  int a[10][10];
  int i,j,row,col;
  struct row_node *first,*p;
  first=NULL;
  printf("Enter the row and cols..\n");
  scanf("%d %d",&row, &col);


  printf("Enter the data for the matrix..\n");
  for(i=0;i<row;i++)
  {
     for(j=0;j<col;j++)
      scanf("%d",&a[i][j]);
  }
  //storing the matrix as a multi list...;

  first=create_rows(row);
  p=first;
  for(i=0;i<row;i++)
```

```
{
   for(j=0;j<col;j++)
     if(a[i][j]!=0)
       insert_list(p,i,j,a[i][j]);
   p=p->next_row;

}
//displaying the matrix as a list

display(first);
}


//creates a linked list to represent the rows
struct row_node* create_rows(int r)
{
 struct row_node *p,*q;
 struct row_node *temp;

 int i;
 p=NULL; // points to the first row node
 q=NULL; // points to the last row node
 //create r number of row nodes
 for(i=0;i<r;i++)
 {
  temp=malloc(sizeof(struct row_node));
  temp->row=i;
  temp->next_row=NULL;
  temp->next_col=NULL;

  if (p==NULL)//first node
  {
    p=temp;
    q=temp;
  }
  else
  {
   q->next_row=temp;
   q=temp;
  }
 }
```

```
  return p;//return the address of the first row node
 }

 void insert_list(struct row_node *p,int row, int col, int x)
 {
   struct col_node *q,*prev,*temp;
   int i,j;


   temp=malloc(sizeof(struct col_node));
   temp->col=col;
   temp->data=x;
   temp->next_col=NULL;

   //insert each column node at the end of the list
   q=p->next_col;
   if(q==NULL)
    p->next_col=temp;
  else
  {
   while(q->next_col!=NULL)
    q=q->next_col;
   q->next_col=temp;
  }
}


 void  display(struct row_node *p)
 {
   struct col_node *q;
   printf("\n");
   while(p!=NULL)
   {
     printf("%d ->",p->row);
     q=p->next_col;
     while(q!=NULL)
     {
      printf("%d,",q->col);
      printf("%d -> ",q->data);
      q=q->next_col;
     }
```

```
    p=p->next_row;
    printf("\n");
   }
 }
```

Program to add two long numbers using circular lists with header node
The numbers are read as an array of characters, later each character converted to a integer

```
#include<stdio.h>
#include<stdlib.h>

 struct node
 {
  int data;
  struct node *next;
 };
  void insert_first(int,struct node*);
  void create_head(struct node **);
  void sum(struct node *,struct node *, struct node *);
 void create_list(char*,struct node*);
 void display(struct node *);
 int main()
 {
  struct node *head1,*head2,*head3;
  char a[10],b[10];
  int x,ch,pos;
  head1=head2=head3=NULL;
  create_head(&head1);//creates the header node
  create_head(&head2);
  create_head(&head3);
  printf("Enter the first number\n");
  scanf("%s",a);
  printf("Enter the second number\n");
  scanf("%s",b);
  create_list(a,head1);
  printf("\n");
  display(head1);
  create_list(b,head2);
  printf("\n");
  display(head2);
```

```
 sum(head1,head2,head3);
 printf("\nsum=");
 display(head3);
}
```

//convert each character into a number and insert into the list
//the list will be created in the reverse order

```
 void create_list(char *a, struct node *h)
 {
    int i=0;
    while(a[i]!='\0')
      {
       insert_first(a[i]-'0',h);
        i++;
      }

  }
```

// create a header node

```
  void create_head(struct node **h)
  {
     struct node *temp;
     temp=(struct node*)malloc(sizeof(struct node));
     temp->data=0;
     temp->next=temp;
      *h=temp;
   }
```

//insert at the head of the list

```
void insert_first(int x, struct node *p)
  {
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=p->next;
    p->next=temp;
   }
```

```
//finds the sum of the two lists
void sum(struct node *h1,struct node *h2, struct node *h3)
{
  struct node *p,*q;
  int sum,carry;
  sum=carry=0;
  p=h1->next;
  q=h2->next;

   while((p!=h1)&&(q!=h2))
   {
     sum=(p->data+q->data+carry)%10;
     carry=(p->data+q->data+carry)/10;
     insert_first(sum,h3);
     p=p->next;
     q=q->next;
    }
   if(p==h1)
   {
     while(q!=h2)
     {
        sum=(q->data+carry)%10;
        carry=(q->data+carry)/10;
        insert_first(sum,h3);
        q=q->next;

      }
   }
   else
     {
      while(p!=h1)
      {
       sum=(p->data+carry)%10;
        carry=(p->data+carry)/10;
        insert_first(sum,h3);
        p=p->next;
       }
     }
   if(carry!=0)
      insert_first(carry,h3);
  }
```

```
void display(struct node *p)
  {
    struct node *q;
    q=p;
    q=p->next;
    while(q!=p)
    {
      printf("->%d ",q->data);
      q=q->next;
     }
   }
```

Program to add two polynomials implemented as a singly linked lists

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>

 struct node
 {
  int coeff;
  int px;
  int py;
  int flag;
  struct node *next;
 };
  void createpoly(struct node **);
  void insert_tail(int,int,int,struct node**);
  void display(struct node*);
  void polyadd(struct node*,struct node*, struct node**);
 main()
 {
  struct node *first,*second,*third;

  first = NULL;
  second=NULL;
  third=NULL;
  int cf,px,py,result;

  printf("\nCreating first polynomial..\n");
```

```
 createpoly(&first);
 printf("\nCreating the second polynomial..\n");
 createpoly(&second);
 printf("\nAdding the two polynomials & displaying the result..\n");
 polyadd(first,second,&third);
 display(third);
}

 void createpoly(struct node **p)
 {
   int cf,px,py;
  while(1)
  {
     printf("\nEnter the coefficient..");
     scanf("%d",&cf);
     if(cf==0)
       break;
     printf("\nEnter the power of x..");
     scanf("%d",&px);
     printf("\nEnter the power of y...");
     scanf("%d",&py);
     insert_tail(cf,px,py,p);
  }

   printf("\nThe polynomial created...\n");
   display(*p);
  }

  void display(struct node *q)
   {
     while(q!=NULL)
     {
        if(q->coeff>0)
         printf(" +%d ",q->coeff);
        else
         printf(" %d ",q->coeff);
        if(q->px>0)
        {
          if(q->px==1)
            printf("X");
          else
```

```
        printf("X^%d",q->px);;
      }
      if(q->py>0)
      {
       if(q->py==1)
        printf("Y");
       else
        printf("Y^%d",q->py);
      }
     q=q->next;
   }
 }


void insert_tail(int cf,int px,int py, struct node **p)
{
  struct node *q,*temp;
  temp=(struct node*)malloc(sizeof(struct node));
  temp->coeff=cf;
  temp->px=px;
  temp->py=py;
  temp->flag=1;
  temp->next=NULL;

  q=*p;
  if(q==NULL)//if it is the first node
    *p=temp;
  else
   {
    while(q->next!=NULL)//go to the last node
      q=q->next;
    q->next=temp;
  }
}

void polyadd(struct node *p,struct node *q,struct node **t)
{
  int x1,y1,cf,c1,x2,y2,c2;
  struct node *q1;
  while(p!=NULL)
  {
```

```
    c1=p->coeff;
    x1=p->px;
    y1=p->py;
    q1=q;
    while(q1!=NULL)
    {
      c2=q1->coeff;
      x2=q1->px;
      y2=q1->py;
      if((x1==x2)&&(y1==y2))
         break;
      q1=q1->next;
     }
     if(q1!=NULL)//still in mid of second poly and found the powers equal
     {
      cf=c1+c2;//add the coefficient
      q1->flag=0;
       if(cf!=0)
         insert_tail(cf,x1,y1,t);//add the sum coeff to the poly
      }
      else
         insert_tail(c1,x1,y1,t);//add the first term to poly;
      p=p->next;
   }

   q1=q;
    while(q1!=NULL)
    {
       if(q1->flag==1)
          insert_tail(q1->coeff,q1->px,q1->py,t);
        q1=q1->next;
    }
}
```

Program to evaluate a polynomial implemented as a linked list

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
```

```
struct node
{
 int coeff;
 int px;
 int py;
 struct node *next;
};
 void insert_tail(int,int,int,struct node**);
 void display(struct node *);
 int polyevaluate(struct node *);
main()
{
 struct node *first;

 first = NULL;
 int cf,px,py,result;
 while(1)
 {
     printf("\nEnter the coefficient..");
     scanf("%d",&cf);
     if(cf==0)
       break;
     printf("\nEnter the power of x..");
     scanf("%d",&px);
     printf("\nEnter the power of y...");
     scanf("%d",&py);
     insert_tail(cf,px,py,&first);
  }
  printf("\nThe polynomial created...\n");
   display(first);
  printf("\nEvaluating the polynomial..\n");
  result=polyevaluate(first);
  printf("Result=%d",result);
 }

  int polyevaluate(struct node *p)
  {
    int x,y,sum;
    printf("\nEnter the value of x and y..");
    scanf("%d %d",&x,&y);
```

```
    sum=0;
    while(p!=NULL)
    {
      sum=sum+(p->coeff*pow(x,p->px)*pow(y,p->py));
      p=p->next;
    }
   return sum;
}

  void display(struct node *q)
   {
     while(q!=NULL)
     {
        if(q->coeff>0)
         printf("+%d",q->coeff);
        else
         printf("%d",q->coeff);
        if(q->px>0)
        {
          if(q->px==1)
            printf("X");
          else
            printf("X^%d",q->px);;
         }
         if(q->py>0)
         {
          if(q->py==1)
           printf("Y");
          else
           printf("Y^%d",q->py);
         }
        q=q->next;
      }
    }



   void insert_tail(int cf,int px,int py, struct node **p)
   {
     struct node *q,*temp;
     temp=(struct node*)malloc(sizeof(struct node));
```

```
  temp->coeff=cf;
  temp->px=px;
  temp->py=py;
  temp->next=NULL;

  q=*p;
  if(q==NULL)//if it is the first node
    *p=temp;
  else
   {
    while(q->next!=NULL)//go to the last node
      q=q->next;
    q->next=temp;
   }
 }
```

References from the Text Book and Reference Book

Book	Chapter	Section	Topic	Page Number
Data Structures using C & C++, Yedidyah Langsam, Moshe J. Augenstein, Aaron M. Tenenbaum , 2nd Edition, 2015	1	1.1	Data structures and C	22
	4	4.2	Linked Lists : Inserting and Removing Nodes from a list	187
			Linked List as Data Structure	195
			Examples of List Operations	198
		4.3	Allocating and Freeing Dynamic Variables	207

			Linked Lists using Dynamic Variables	**215**
			Examples of List operations in C	
		4.5	**Circular Lists**	**229**
			Addition of Long positive integers using Circular Lists	**235**
			Doubly linked Lists	**237**
			Addition of Long positive integers using Doubly Linked Lists	**239**
Data Structures and Program Design in C, Robert L. Kruse, Clovis L. Tando, Bruce P. Leung, 2nd Edition, 2007	**4**	**4.5**	**Pointers and Linked List**	**151**
	5	**5.1**	**List Specifications**	**185**
		5.2	**Implementation of Lists**	**187**
		5.4	**Application : text Editor**	**201**
	7	**7.2.1**	**Ordered Lists**	**272**

Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

CIRCULAR DOUBLY LINKED LIST

Abstract

Circular Double Linked List – overview
and pseudo code of various operations

Vandana M Ladwani
vandanamd@pes.edu

Contents

Overview

A circular doubly linked list has both successor pointer and predecessor pointer in circular manner that is the last and first node are connected. The objective behind considering circular double linked list is to simplify the insertion and deletion operations performed on double linked list. In circular double linked list the rlink link of the right most node points back to the head node and llink link of the first node points to the last node.

The basic operations in a circular double linked list are:

- Creation
- Insertion
- Deletion
- Traversing

Operations

Inserting a node at the beginning

The following steps are to be followed to insert a new node at the beginning of the list:

- Get the new node using createnode().
  new_node=createnode();
- If the list is empty, then
  head = new_node;
  new_node -> llink = head;
  new_node -> rlink = head;
- If the list is not empty, follow the steps given below:
  new_node -> llink = head -> llink;
  new_node -> rlink = head;
  head -> llink -> rlink = new_node;
  head -> llink = new_node;
  head = new_node;

Inserting a node at the end

The following steps are followed to insert a new node at the end of the list:

- Get the new node using createnode()
  new_node=createnode();
- If the list is empty, then
  head = new_node;

new_node -> llink = head;
new_node -> rlink = head;

- If the list is not empty follow the steps given below:
  new_node -> llink = head -> llink;
  new_node -> rlink = head;
  head -> llink -> rlink = new_node;
  head -> llink = new_node;

Inserting a node at an intermediate position

The following steps are followed, to insert a new node in an intermediate position in the list:

- Get the new node using createnode().
  new_node=createnode();
- Ensure that the specified position is in between first node and last node. If not, specified position is invalid. This is done by countnode() function.
- Store the heading address (which is in head pointer) in temp. Then traverse the temp pointer upto the specified position.
- After reaching the specified position, follow the steps given below:
  new_node -> llink = temp;
  new_node -> rlink = temp -> rlink;
  temp -> rlink -> llink = new_node;
  temp -> rlink = new_node;
  nodectr++;

Deleting a node at the beginning

The following steps are followed, to delete a node at the beginning of the list:

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given below:
  temp = head;
  head = head -> rlink; // second node
  temp -> llink -> rlink = head;// temp->llink is last node
  head -> llink = temp -> llink;
  free(temp)

Deleting a node at the end

The following steps are followed to delete a node at the end of the list:

- If list is empty then display 'Empty List' message
- If the list is not empty, follow the steps given below:

```
cur=head->llink          // last node address
cur->llink->rlink=head // cur->llink is second last node
head->llink=cur->llink
free(cur)
```

Deleting a node at Intermediate position:

The following steps are followed, to delete a node from an intermediate position in the list (List must contains more than two node).

- If list is empty then display 'Empty List' message.
- If the list is not empty, follow the steps given below:
- Get the position of the node to delete.
- Ensure that the specified position is in between first node and last node. If not, specified position is invalid.
- Then perform the following steps:

```
if(pos > 1 && pos < nodectr)
    temp = head;
    i = 1;
    while(i < pos)
        temp = temp -> rlink ;
        i++;
    temp -> rlink -> llink = temp -> llink;
   temp -> llink -> rlink = temp -> rlink;
   free(temp);
   nodectr--;
```

Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

SPARSE MATRIX

Abstract

SPARE MATRIX – overview and its representations

Vandana M Ladwani

vandanamd@pes.edu

Contents

SPARSE MATRIX

Overview

A matrix is represented as 2 dimensional array where every element is accessed by row and column index. Real world data such as image, spectrogram and graph can be modelled using matrices.
A matrix for which most of values are zero is termed as the sparse matrix. If a sparse matrix is stored in a memory as a two dimensional matrix it wastes lot of space.
So alternate representations are preferred for sparse matrix
Alternate representations are:

- Triple notation
- Linked representation

Representations

1. Triple notation

In triple notation sparse matrix is represented as an array of tuple values. Each tuple consists of
<rowno columnno Value>

The first block in array block holds information regarding
<total no of rows, total no of columns ,value>

Declaration

```
typedef struct
{
        int col;
        int row;
        int value;
} term;
term a[10];
```

Various operations that can be performed on sparse matrix are
Create_SparseMatrix()
Transpose_of_SparseMatrix()
Add_SparseMatrices()

Multiple_SparseMatrices()

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 4 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \\ 8 & 0 & 0 & 1 \\ 0 & 0 & 6 & 0 \end{bmatrix}$$

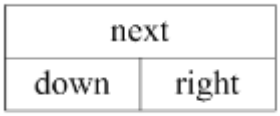
Triple Notation

Row No	Column No	Value
5	4	6
0	0	2
1	0	4
1	3	3
3	0	8
3	3	1
4	2	6

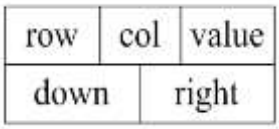
2. Linked Representation

Two types of nodes are used

Header Node

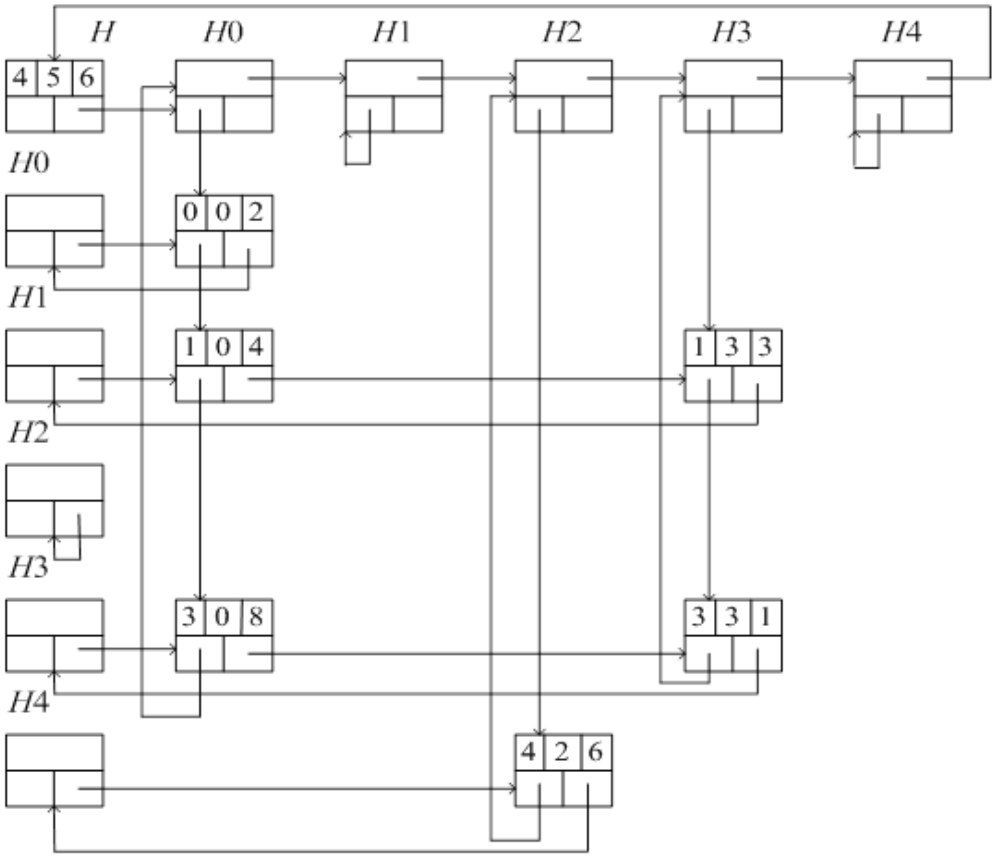


Data Node




```
#define MAX_SIZE 50 /* size of largest matrix */
typedef enum {head, entry} tagfield;
typedef struct matrixNode * matrixPointer;
typedef struct entryNode {
int row;
int col;
int value; };
typedef struct matrixNode {
        matrixPointer down;
        matrixPointer right;
        tagfield tag;
        union
        {
        matrixPointer next;
        entryNode entry;
        } u;
};
```

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 4 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \\ 8 & 0 & 0 & 1 \\ 0 & 0 & 6 & 0 \end{bmatrix}$$



Sparse Matrix representation using Linked Nodes

Courtesy: “Fundamentals of Data Structures” By Ellis Horowitz and Sartaj Sahni

PES University
Department of Computer Science and Engineering

UE19CS202- Data Structures and its Applications(4-0-0-4-4)

UNIT - 2 : STACKS AND QUEUES

Note : Pointers, Arrays, Structures, Linked List to be revised for clear understanding.

Course Content for Unit 2

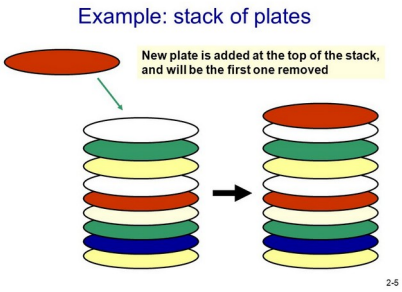
Stacks: Basic structure of a Stack, Implementation of a stack using Arrays & Linked list. **Applications of Stack:** Function execution, Nested functions, Recursion : Tower of Hanoi. Conversion & Evaluation of an expression: Infix to postfix, Infix to prefix, Evaluation of an Expression, Matching of Parentheses. **Queues & Deque:** Basic Structure of a Simple Queue, Circular Queue, Priority Queue, Deque and its implementation using Arrays and Linked List. **Applications of Queue: Case Study – Josephus problem, CPU scheduling- Implementation using queue(simple /circular).**

Contents :

6. Queues
7. Basic Structure of Queue
8. Implementation of queue using Arrays and Linked List
9. Circular queue
10. Priority Queue
11. Application of Queues
12. Case Study
13. References
14. Sample Code.

1. Introduction to Stacks:

- Consider a restaurant serving buffet lunch / dinner where plates are placed one above another. Every new plate is placed on the top.When a plate is required by a customer, it is taken off from the top and used. This is called stacking of plates.
- Consider another example of a driveway in a parking lot in a small apartment complex where all the cars will be parked as they enter and the new car will be parked at the last. To take out the car from the parking area, the last car which was parked should be taken out.
- In the above 2 scenarios, it is observed that the last plate added on top or the last car parked at the parking lot will be taken out first.
- Using this analogy, stack is defined as a linear data structure where items are inserted from one end called the **top** of the stack and removed from the same end.
- Last item inserted will always be on the top of the stack. Deletion is also done from the same end.
- Last item inserted first is the first item removed. Hence Stack is called **Last In First Out (LIFO) Data Structure.**



2. STACK - AS ADT

2 major operations that can be performed on stack.

- Insert a element into the stack - void push ()
- Delete an element from stack -int pop ().

2. 1 INSERTION OPERATION ON A STACK

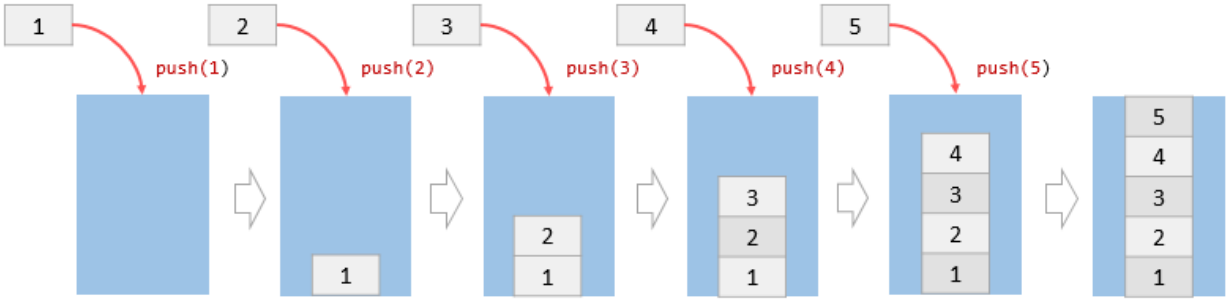


Figure 1 : Stack insertion operation

- Inserting an element into stack is called as PUSH Operation
- From the above figure, it is seen that the stack is fixed with size 5 and initially the stack is empty.
- Five elements : 1,2,3,4,5 are to be placed on the stack.
- Elements are inserted as shown above.
- Initially since the stack is empty, **top**(variable) points to the bottom of the stack.
- The first element 1 is pushed to the stack. At the same time top is incremented by 1.
- The second element 2 is pushed onto the stack, the top is incremented.
- Each time the elements are inserted, top is incremented.
- Similarly elements 3,4,5 are inserted one after the other on the stack.
- After inserting 5, the stack is full and there is no space left to insert.
- The situation in which stack is full and not possible to insert any new item is called **Stack Overflow or Stack FULL.**

2.2 DELETION OPERATION ON A STACK - POP

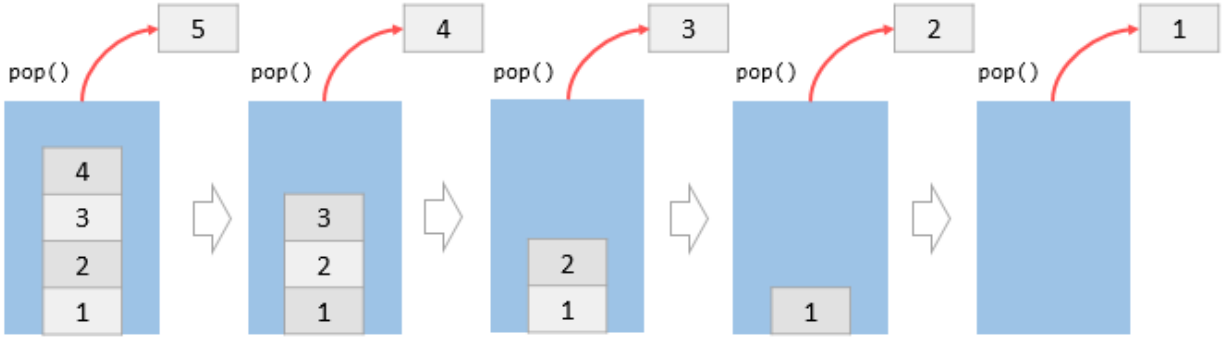


Figure 3 : Deletion an element from stack.

- Deletion of element from the stack is called as POP OPeration
- Deletion of elements from stack should be done from the same end as insertion.
- Elements are deleted from the top of the stack.
- Once the top most element is deleted from stack, the element below it becomes the new top element.
- Each time an element is decremented from the stack, the top is decremented.
- From the above diagram, it can be seen that the first element popped from the stack is 5. Also in parallel top is also decremented to the next highest position.
- After the elements 4,3,2,1 are removed, the top is decremented to the bottom of the stack and the stack is empty.
- Situation when the stack is empty and not possible to delete anymore elements is called **Stack Empty or Stack Underflow.**

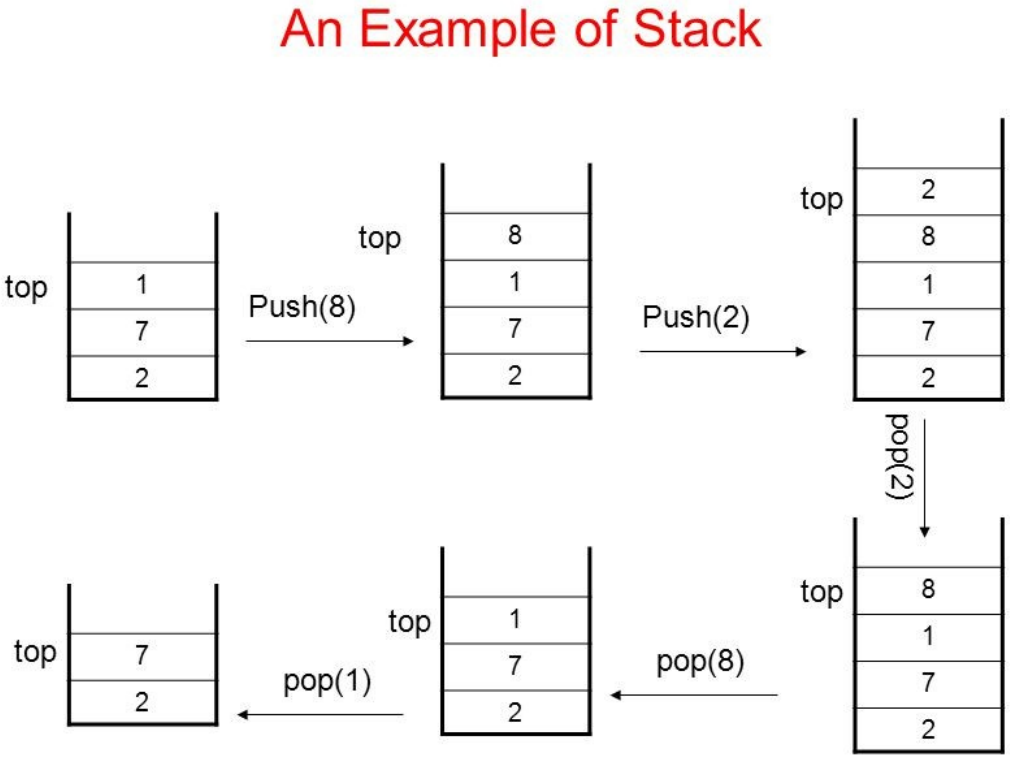
3 IMPLEMENTATION OF STACK

Stack Data structure can be implemented in two ways:

1. Using Arrays.
2. Using Linked List.

3.1 Implementation using Arrays:

- Stack is implemented using 1D Array.
- When a stack is implemented using arrays, its size remains fixed. It is not possible to increase or decrease the size.
- Define a 1D array to insert / delete using LIFO principle, initialize a variable top to -1.
- Whenever we want to insert a value into the stack, increment the top value by one and then insert.
- Whenever we want to delete a value from the stack, then delete the top value and decrement the top value by one.



Following steps are to be followed for creating empty stack:

1. Create a 1- D Array with fixed size
2. Initialize a variable top to -1.

Insert an element into stack - push()

- Push() is a function used to insert an element into the stack.
- In a stack, the new element is always inserted at top position.
- Push function takes one integer value as a parameter and inserts that value into the stack.

  Step 1 - Check whether stack is FULL. (top == SIZE-1)

  Step 2 - If it is FULL, then display "Stack overflow" and terminate the function.

  Step 3 - If it is NOT FULL, then increment top value by one (top++) and set stack[top] to value (stack[top] = value)

Delete an element from stack - pop()

- pop() is a function used to delete an element from the stack.
- The element is always deleted from the top position.

- Pop function does not take any value as a parameter.

  Step 1 - Check whether stack is EMPTY. (top == -1)

  Step 2 - If it is EMPTY, then display "Stack Underflow!" and terminate the function.

  Step 3 - If it is NOT EMPTY, then delete stack[top] and decrement top value by one (top--)

Display the contents of Stack- display()

Step 1 - Check whether stack is EMPTY. (top == -1)

Step 2 - If it is EMPTY, then display "Stack is EMPTY!!!" and terminate the function.

Step 3 - If it is NOT EMPTY, then define a variable 'i' and initialize it with top. Display stack[i] value and decrement i value by one (i--).

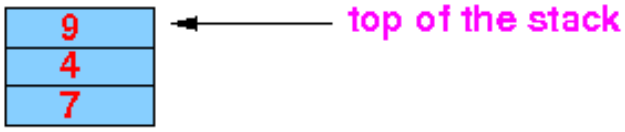
Step 4 - Repeat above step until i value becomes '0'.

3.2 Implementation of Stack Using Linked List

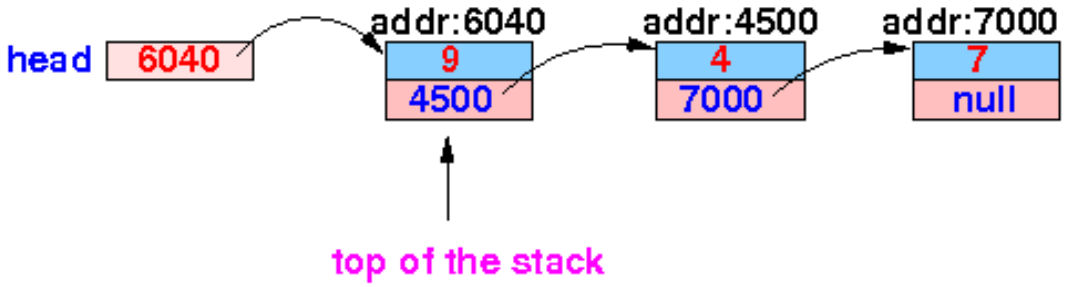
- Drawback of stack using arrays is that it works only with a fixed number of elements.i.e the size should be defined in the beginning itself.
- THis way of implementation is not suitable when the size of the elements are unknown
- To overcome the above drawback, stack can be implemented using the linked list
- Using a linked list based implementation, the size of stack is unlimited, so the data size can be variable.
- There is no need to fix the size initially.
- Here, every new element(Node) is inserted as the **top** in the list. Top is incremented as the node keeps increasing. Similarly as we delete the node, top is decremented.

Representing a stack with a list:

Stack:



List:



Stack Operations using Linked List:

Step 1 - Define a 'STACK' structure with two members - data of type integer and next as type Node.

Step 2 - Define a Node pointer 'top' and set it to NULL or -1.

Inserting an element into a Stack

Step 1- Create an empty node s of type STACK and assign any value.

Step 2 - Check if the stack is empty

Step 3 - If it is Empty, then set s→ next = NULL.

Step 4 - If it is Not Empty, then set p->top++;

p->s[p->top]=x;

Step 5 - Finally, set top = s

Deleting an Element from Stack - POP()

Step 1 - Check if the stack is empty (top == NULL).

Step 2 - If it is Empty, then display "Stack underflow", terminate the functio.

Step 3 - If it is Not Empty, then define a Node pointer 'q' and set it to 'top'.

Step 4 - Then set 'top = top → next'.

Step 5 - Finally, delete q. (free(temp)) / or return q (return type is int)

Display the contents of the Stack

Step 1 - Check whether stack is Empty (top == NULL).

Step 2 - If it is Empty, then display 'Underflow' and terminate the function.

Step 3 - If it is Not Empty, then define a STACK pointer 'stk' and initialize it with top. Traverse the list by starting from top till u reach the end of list

Step 4 - Display 'stk → data --->' and move it to the next node. Repeat the same until stk reaches to the first node in the stack. (stk → next != NULL).

Step 5 - Finally display the list is empty.

4. APPLICATIONS OF STACK

There are various applications of stack:

- Conversion of Expression.
- Evaluation of Expression.
- Recursion.
- Parentheses Checking.
- Syntax Parsing.
- Backtracking - gaming, finding path.

4.1 CONVERSION OF AN EXPRESSION

- **Expression** : An expression is a mathematical statements consisting of the operators between the operands
- It can be syntactically defined as follows:
  - **A op B**.
- Here A and B are called operands and op is the operator.
- Operators are mainly used to perform mathematical operations between the operands.
- Operators can be any mathematical operators like - Addition, Subtraction, Multiplication and Division.

Types of Expressions:

1. Infix Expressions
2. Prefix Expressions
3. Postfix Expressions.

Infix Expressions: Expression having an operator between two operands

1. a + b * c
2. (2+2)*(4/1)

Prefix expressions : Expressions where the operators precede the operands

1. +a*bc
2. *+22/14

Postfix Expressions: Expressions where the operators succeed the operands

1. bc*a+
2. 41/22+/

4.1 CONVERSION OF INFIX TO POSTFIX EXPRESSION

1. In the given infix expression fill all the operators
2. As per the operator precedence, operators are evaluated accordingly.
3. Convert into the respective postfix form in the same order of evaluation.

Steps in converting infix expression to postfix

1. Scan all the input symbols(operands & Operators) from L-R from the infix expression.
2. If the input symbol is operand, then output and print the operand directly.
3. If the input symbol is left parenthesis '(',push it onto the Stack.
4. If the input symbol is right parenthesis ')', then Pop all the contents of stack until matching left parenthesis is popped and print each popped symbol to the result.
5. If the input symbol is operator i.e + , - , * , / , then Push it onto the Stack.
6. If first pop the operators which are already on the stack that have higher or equal precedence than current operators and print them to the result.
7. Resultant expression will be in the postfix form - AB op where A and B are operands and op is the operator.

Symbol	**Current Operator Precedence**	**Stack Precedence**
+, -	1	2
*, /	3	4
Operands	7	8
(	9	0
)	0	-
Top of Stack Initially (#)	-	-1

4.2 EVALUATION OF POSTFIX EXPRESSIONS

- In the previous section, we discussed the conversion of the infix expression to its postfix form.
- In a postfix expression, operator suffixes the operands.
- General Structure of a postfix expression is
  - A B op
- Here, A and B are operands and op is the operator.

Evaluation of postfix Expression using Stack

Following steps are followed while evaluating the postfix expression.

1. Create an empty stack - to store operands.
2. Scan the symbols in the given postfix expression.
3. If the input is operand, then push it onto the Stack.
4. If the input is operator (+ , - , * , / etc.,), then the top two operands are popped and store them as 2 different variables.
5. Then perform scanning the symbol operation using operand1 and operand2 and push the result back on to the Stack.
6. Lastly pop and display the popped value as the final result.

Evaluation of Postfix Expression

Postfix → a b c * + d - Let, a = 4, b = 3, c = 2, d = 5

Postfix → 4 3 2 * + 5 -

Operator/Operand	Action	Stack
4	Push	4
3	Push	4, 3
2	Push	4, 3, 2
*	Pop (2, 3) and 3*2 = 6 then Push 6	4, 6
+	Pop (6, 4) and 4+6 = 10 then Push 10	10
5	Push	10, 5
-	Pop (5, 10) and 10-5 = 5 then Push 5	5

www.geekyshows.com

4.3 PARENTHESIS MATCHING

- Parentheses match also known as balancing of the parentheses is a common coding syntax error that occurs during early programming.
- The following are examples of matching parentheses - ({}) (())({}() ()
- The following are examples of non matching parentheses - {} ({}( ){}) (()
- Following are the steps to the followed while matching the parentheses

Step - 1 -Scan the input string.

Step -2 If character is opening parenthesis `'(', ‘{‘,’[‘`, push it on stack.

Step - 3 If character is closing parenthesis `')',’}’,’]’.`

Step 3.1 Check top of stack, if it is `'(',’{‘,’[‘` , pop and move to next character.If it is not `'('`, return false.

Step - 4 After scanning the entire string, check if the stack is empty. If stack is empty, return true else return false.

5. RECURSION

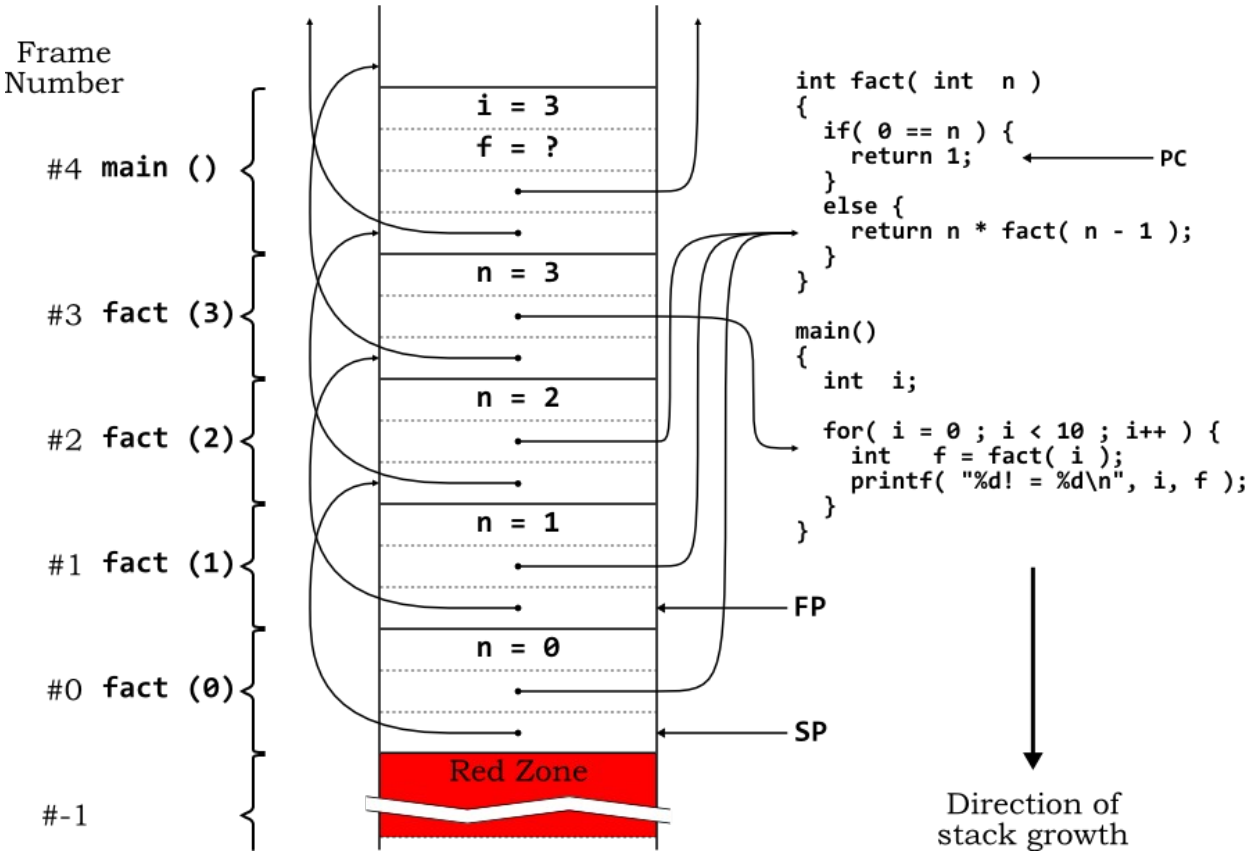
- Powerful utility available in most of the programming languages- C, C++, python, Java etc..
- Recursion is a method of solving problems where the solution depends on smaller instances of the same problem.
- In recursion, a function ‘a’ either calls itself directly or calls a function ‘b’ that in turn calls the original function ‘a’.
- A recursive function using has 2 cases:
    - A base case
    - A Recursive case
- A base case in recursion, in which the answer is known when the termination for a recursive condition is to unwind back.
- A recursive case which returns to the answer which is closer

RECURSION Vs ITERATION

Diff between Recursion & Iteration:

Recursion in Data Structures - Implementation of Recursion

- Implementation recursion by means of Stacks Data Structure.
- Whenever a function (caller) calls another function (callee) or itself as callee, the caller function transfers execution control to the callee.
- This transfer process may also involve some data to be passed from the caller to the callee.
- This implies, the caller function has to suspend its execution temporarily and resume later when the execution control returns from the callee function.
- The caller function needs to start exactly from the point of execution where it puts itself on hold.
- It also needs the exact same data values it was working on.
- For this purpose, an activation record (or stack frame) is created for the caller function.



The above diagram shows the stack implementation of a factorial program executed recursively.

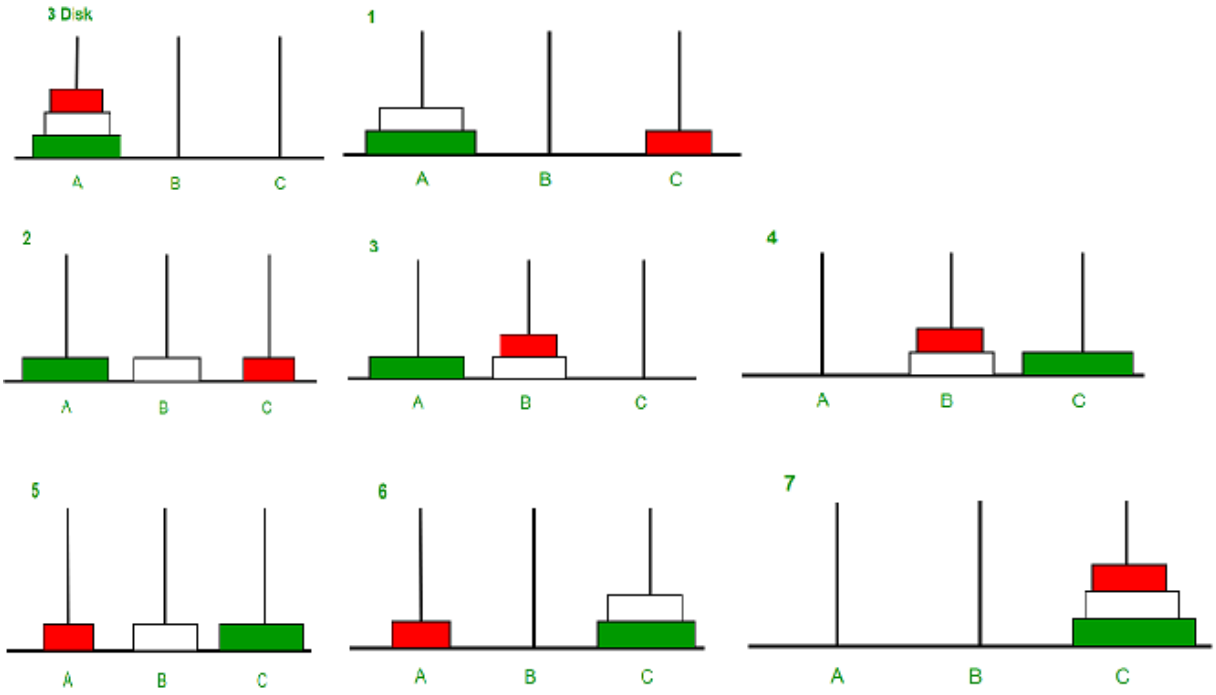
Example : Tower of Hanoi

It is a classical mathematical puzzle where there are three rods and n disks. The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

1) Only one disk can be moved at a time.

2) Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.

3) No disk may be placed on top of a smaller disk



In the above diagram there are 3 rods named A, B, C consisting of 3 disks(Orange 'O' at Top, White 'W' at middle and Green 'G' at Bottom) in Rod A.
Step 1 : Move 'R' to C.
Step 2 : Move 'W' to B.
Step 3 : Move 'R' to B.
Step 4 : Move 'G' to C.
Step 5 : Move 'R' to A.
Step 6 : Move 'W' to C.
Step 7 : Move 'R' to C.

Recursive ALgorithm : disk - # of disks, src - source rod, aux - auxiliary rod, destination- destination rod.

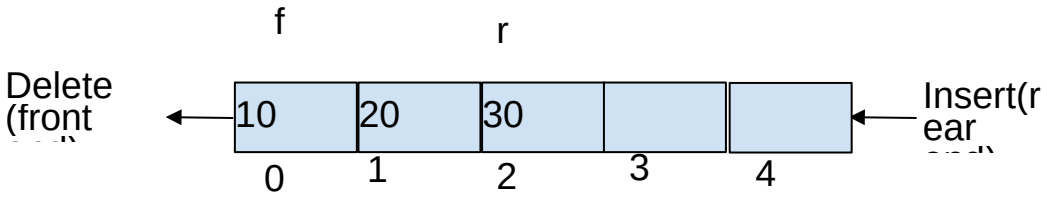
Start
TOH(disk,src,aux).
    If disk == 1 , move from src to destination.
Else
    TOH(disk-1,src,aux,destination). (**Step 1**)
    Move from src to destination. (**Step 2**)
    TOH(disk-1,aux,destination, src). **(Step 3**)

6. QUEUES

In the previous section we discussed about one of the common linear data structures and its applications - Stacks

- In this topic we will be discussing another linear Data Structures and its application and implementation - **Queues**
- Queue is a very familiar term used in day to day life as we see people standing in queues to pay bills, to board buses, to buy tickets in cinema halls etc..
- In these scenarios, a person just enters the queue and stands at the end and the person who is at the front of the first person to pay the bill or get inside the bus or purchase the tickets etc..

- The queue data structure is implemented in the same concept
- A **Queue** is a linear data structure where the elements are inserted from one end and deleted from the other end.
- End from which an element is inserted is called the **rear** end and the end from which the element is deleted is called the **front** end.



Pictorial representation of Queues is as shown above :

- The first element inserted into the queue is 10, the second element inserted in 20 and the last element inserted is 30.
- Any new element to be inserted into the queue has to be inserted towards the right of 30and that element will be the last element in the queue.

- From the above operations on the queue, it can be clearly understood that the first element inserted into the queue is the first element to be deleted out from the queue.
- Hence Queue is known as the **First in First Out (FIFO) Data Structure.**

Applications of Queue in Computer Science:

- CPU Scheduling.
- Time sharing
- Batch Processing

Types of Queue:

1. Ordinary Queue
2. Circular Queue
3. Double Ended Queue
4. Priority Queue

QUEUE - ADT

Operations on Queue

1. Insert into an element from queue - En-queue
2. Delete an element from the queue - Dequeue

Implementation

A queue can be implemented in Two ways :

1. Using Arrays
2. Using LInked List

IMPLEMENTATION OF QUEUE USING ARRAYS:

- Queue can be implemented using 1D Array.
- The queue implemented using an array stores only a fixed number of data values.
- Implementation s very simple -
- Define a one dimensional array of specific size and insert or delete the values into that array by using FIFO mechanism with the help of variables 'front' and 'rear'.
- Initially both 'front' and 'rear' are set to -1.
- Whenever, we want to insert a new value into the queue, increment 'rear' value by one and then insert at that position.
- Whenever we want to delete a value from the queue, then delete the element which is at 'front' position and increment 'front' value by one.

INSERTING AN ELEMENT - ENQUEUE

EnQueue() is a function used to insert a new element into the queue.

The new element is always inserted at the rear position.

- **Step 1** - Check whether the queue is FULL. (rear == SIZE-1)
- **Step 2** - If it is FULL, then display "Queue Overflow!!!" and terminate the function.
- **Step 3** - If it is NOT FULL, then increment rear value by one (rear++) and set queue[rear] = value.

DELETING THE ELEMENT FROM QUEUE

DeQueue() function used to delete an element from the queue.

The element is always deleted from the front position.

The deQueue() function does not take any value as parameter.

- **Step 1** - Check whether the queue is **EMPTY**. (**front == rear**)
- **Step 2** - If it is **EMPTY**, then display **"Queue Underflow "** and terminate the function.
- **Step 3** - If it is **NOT EMPTY**, then increment the **front** value by one (**front ++**). Then display **queue[front]** as a deleted element. Then check whether both **front** and **rear** are equal (**front** == **rear**), if it **TRUE**, then set both **front** and **rear** to '**-1**' (**front** = **rear** = **-1**).

Queue Using Linked List

- Drawback of queues using arrays is that it will work for an only fixed number of data values.
- That is the amount of data or the size of the queue must be specified at the beginning itself.
- Queue using an array is not suitable when the size of the data is too large or unknown.
- To overcome the above drawback, queue can be implemented using a linked list data structure.
- A Linked list can work for an unlimited number of values- no need to fix the size at the beginning of implementation.
- The Queue implemented using linked lists can organize as many data values as we want.

Operations:

To implement a queue using a linked list, we need to set the following things before implementing actual operations.

Step 1 - Define a '**Node**' structure with two members **data** and **next**.

Step 3 - Define two **Node** pointers '**front**' and '**rear**' and set both to **NULL**.

Step 4 - Implement the **main** function by displaying a menu of list of operations and make suitable function calls in the **main** method to perform user selected operation.

ENQUEUE - INSERTION ELEMENT FROM FRONT

Step 1 - Create a node called **temp** with given value and set '**temp → next**' to **NULL**.

Step 2 - Check whether queue is **Empty** (**rear** == **NULL**)

Step 3 - If it is **Empty** then, set **front** = **rear=temp**

Step 4 - If it is **Not Empty** then, set **rear → next** = **temp** and **rear** = **temp.**.

DE-QUEUE - DELETION FROM FRONT END

Step 1 - Check whether the queue is Empty **(front == NULL).**

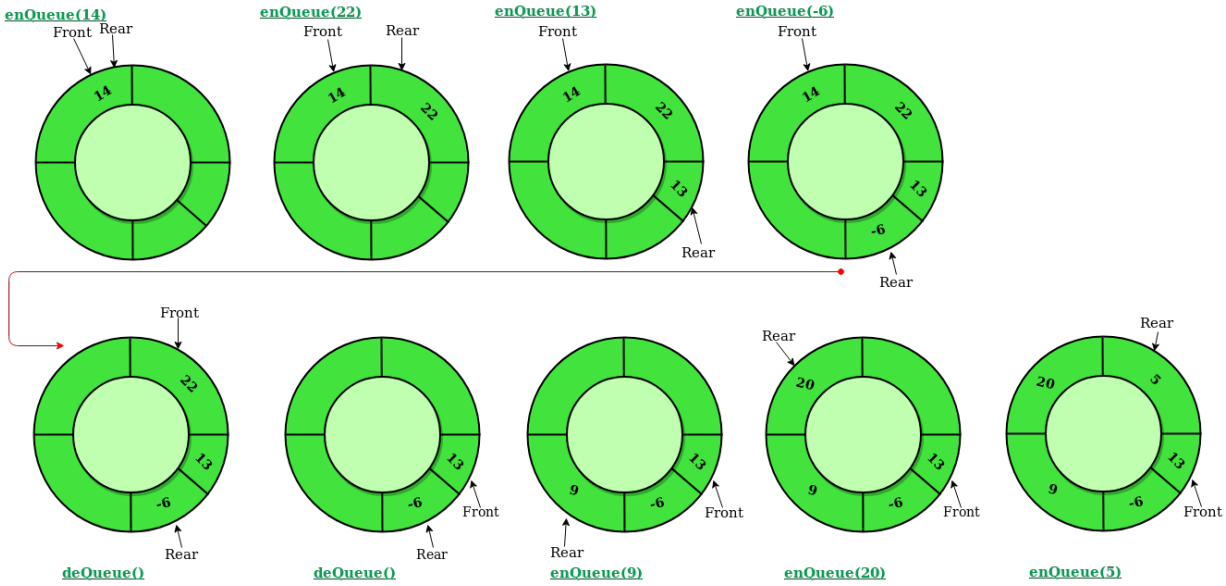
Step 2 - If it is Empty, then display "Queue underflow" and terminate from the function

Step 3 - If it is Not Empty then, define a Node pointer 'temp' and set it to 'front'.

Step 4 - Then set 'front = front → next' and delete 'temp' (free(temp)).

CIRCULAR QUEUE

- In an ordinary queue, elements are inserted, the rear end identified by ‘r’ is incremented by 1.
- Once the queue reaches MAX-1 , the queue is full.
- Even If some elements are deleted from the queue, and rear = MAX-1, items cannot be inserted.
- To overcome the above drawback, a circular queue is used.
- A circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle.



IMPLEMENTATION OF CIRCULAR QUEUE

Step 1 - Declare all functions used in circular queue implementation.(insert, delete,display)

Step 2 - Create a one dimensional array with a predefined size.

Step 3 - Define two integer variables 'front' and 'rear' and initialize both with '-1'.

Step 4 - Implement the main method by displaying a menu of operations list and make suitable function calls to perform operations selected by the user on a circular queue.

Enqueue - Inserting value into circular Queue

- EnQueue() is a function used to insert an element into the circular queue.
- In a circular queue, the new element is always inserted from its rear position.

Step 1 - Check whether the queue is FULL. ((rear == SIZE-1 && front == 0) || (front == rear+1))

Step 2 - If it is FULL, then display "Queue overflow" and terminate the function.

Step 3 - If it is NOT FULL, then check rear == SIZE - 1 && front != 0 if it is TRUE, then set rear = -1.

Step 4 - Increment rear value by one (rear++), set queue[rear] = value and check 'front == -1' if it is TRUE, then set front = 0.

Dequeue - Deleting value into circular Queue

- DeQueue() is used to delete an element from the circular queue.
- In a circular queue, the element is always deleted from the front position.

Step 1 - Check whether the queue is **EMPTY**. (**front == -1 && rear == -1**)

Step 2 - If it is **EMPTY**, then display **"Queue underflow"** and terminate the function.

Step 3 - If it is **NOT EMPTY**, then display **queue[front]** as a deleted element and increment the **front** value by one (**front ++**). Then check whether **front == SIZE**, if it is **TRUE**, then set **front = 0**. Then check whether both **front - 1** and **rear** are equal (**front -1** == **rear**), if it **TRUE**, then set both **front** and **rear** to '**-1**' (**front** = **rear** = **-1**).

DOUBLE ENDED QUEUE

Double ended queue is a special type of data structure in which insertions and deletions will be done either at front end or at rear end of the queue.

Operations of Double ended queue

1. Insert an element from the front end.
2. Insert an element from the rear end.
3. Delete an element from the front end.
4. Delete an element from the rear end.
5. Display the contents of the queue.

Implementation: Assignment for Students

PRIORITY QUEUE

- A priority queue is a special type of queue.
- In a priority queue elements are ordered by key value so that item with the lowest value of key is at front and item with the highest value of key is at rear or vice versa.
- So we're assigned priority to the element based on its key value.
- Lower the value, higher will be the priority.

APPLICATIONS OF PRIORITY QUEUE

1. Dijskstra's shortest path algorithm
2. Prims Algorithm
3. Artificial intelligence - A Search Algorithm.

IMPLEMENTATION:

Following are the steps used in the implementation of Priority queue using a list -

INSERTION

Step 1 : If there is no new node, create a node dynamically to allocate memory.

Step 2 : if the node is already present, insert the new node at the end from left to right.

Step 3 : Heapify the array.

DELETION

Step 1 : If the node to be deleted is leaf node, remove the node

Step 2 : Swap the node to be deleted with the last leaf node.

Step 3 : Heapify the array.

Priority Queue Applications

Some of the applications of a priority queue are:

1. Dijkstra's algorithm
2. for implementing stack
3. for load balancing and interrupt handling in an operating system
4. for data compression in Huffman code

SAMPLE CODE :

Implementation of Stacks to perform push and pop operations:

```
#include<stdio.h>
#include<stdlib.h>
int push(int*,int*,int,int);
int pop(int*,int*);
void display(int*,int);
//driver function.
int main()
{
  int top,size,ch,k,x;
  int *s;
  printf("Enter the size of the stack..\n");
  scanf("%d",&size);
  s=(int*)malloc(sizeof(int)*size);
  top=-1;
  while(1)
  {
    display(s,top);
    printf("\n1..push\n");
    printf("2..pop\n");
    printf("3..display\n");
    scanf("%d",&ch);
    switch(ch)
    {
      case 1:printf("Enter the data\n");
             scanf("%d",&x);
             k=push(s,&top,size,x);
             if(k>0)
```

```
            printf("Element pushed successfully\n");
            break;
    case 2:k=pop(s,&top);
           if(k>0)
            printf("\nElement popped=%d\n",k);
           break;
     case 3:display(s,top);
           break;
     case 4:exit(0);
   }
 }
}
//pop
 int pop(int *s, int *t)
 {
   int x;
   //check for stack underflow
   if(*t==-1)
   {
     printf("Stack underflow..cannot delete..\n");
     return -1;
   }
    x=s[*t];
    (*t)--;
    return x;
  }
//push
 int push(int *s, int *t, int size, int x)
 {
    //check for overflow
```

```
    if(*t==size-1)
    {
      printf("Stack overflow..cannot insert\n");
      return -1;
     }
     (*t)++;//or ++*t or *t=*t+1
      s[*t]=x;
     return 1;
   }
//display the contents of stack
  void display(int *s, int t)
  {
    int i;
    if(t==-1)
      printf("Empty stack..\n");
    else
     {
       for(i=t;i>=0;i--)
          printf("%d  ",s[i]);
      }
  }
```

Implementation of QUEUE to perform enqueue and dequeue operations

```
#include<stdlib.h>
#include<stdio.h>

int qinsert(int,int*,int*,int*,int);
 int qdelete(int *,int *, int*);
 void display(int *,int,int);
int main()
{

  int *q;
  int ch,k,x;
   int f,r, size;
  f=r=-1;
  printf("Enter the size of the size..");
  scanf("%d",&size);

  q=(int*)malloc(sizeof(int)*size);
  while(1)
  {
    display(q,f,r);
    printf("\n1..Insert");
    printf("\n2..Delete");
    printf("\n3..Display");
    printf("\n4..EXIT");
    scanf("%d",&ch);
    switch(ch)
    {
       case 1:printf("Enter the value..");
              scanf("%d",&x);
```

```
            k=qinsert(x,q,&f,&r,size);
            if(k>=0)
              printf("Element inserted successfully\n");
             break;
       case 2:k=qdelete(q,&f,&r);
            if(k>=0)
              printf("element deleted=%d\n",k);
            break;
        case 4:exit(0);
  }
 }
}

 int qinsert(int x,int *q,int *f,int *r,int size)
 {
  //check for queue overflow

   if(*r==size-1)
   {
     printf("Queue full..\n");
     return -1;
   }
    (*r)++;
    q[*r]=x;
    if(*f==-1)//first element
      *f=0;//point to first element
    return 1;
  }
```

```
int qdelete(int *q,int *f, int *r)
{
   int x;
   //check for queue empty
  if(*f==-1)
  {
     printf("Queue empty..\n");
      return -1;
   }
   x=q[*f];
   if(*f==*r)//only one element
     *f=*r=-1;
   else
     (*f)++;
   return x;
}


 void display(int *q, int f, int r)
 {
    int i;
    if(f==-1)
      printf("Empty Queue..\n");
    else
     {
       for(i=f;i<=r;i++)
          printf("%d  ",q[i]);
     }
    printf("\n");
}
```